Computer Architecture
CS 622 / EE 520
Fall 2026

Lecture 21

Shahid Masud

Topics

- Look at New Processor Architectures
- Page Fault in Virtual Memory
- Memory Architecture
  - Technology
  - Controllers for Cache, Memory and MMU
  - Viewing Cache and TLB together

Quiz Next Monday

Further Readings

Reading Notes

Readings:
Chapter 2 of course textbook

Slide credits

Lecture 11: Memory Hierarchy
-- Cache Organization and Performance

CSCE 513 Computer Architecture

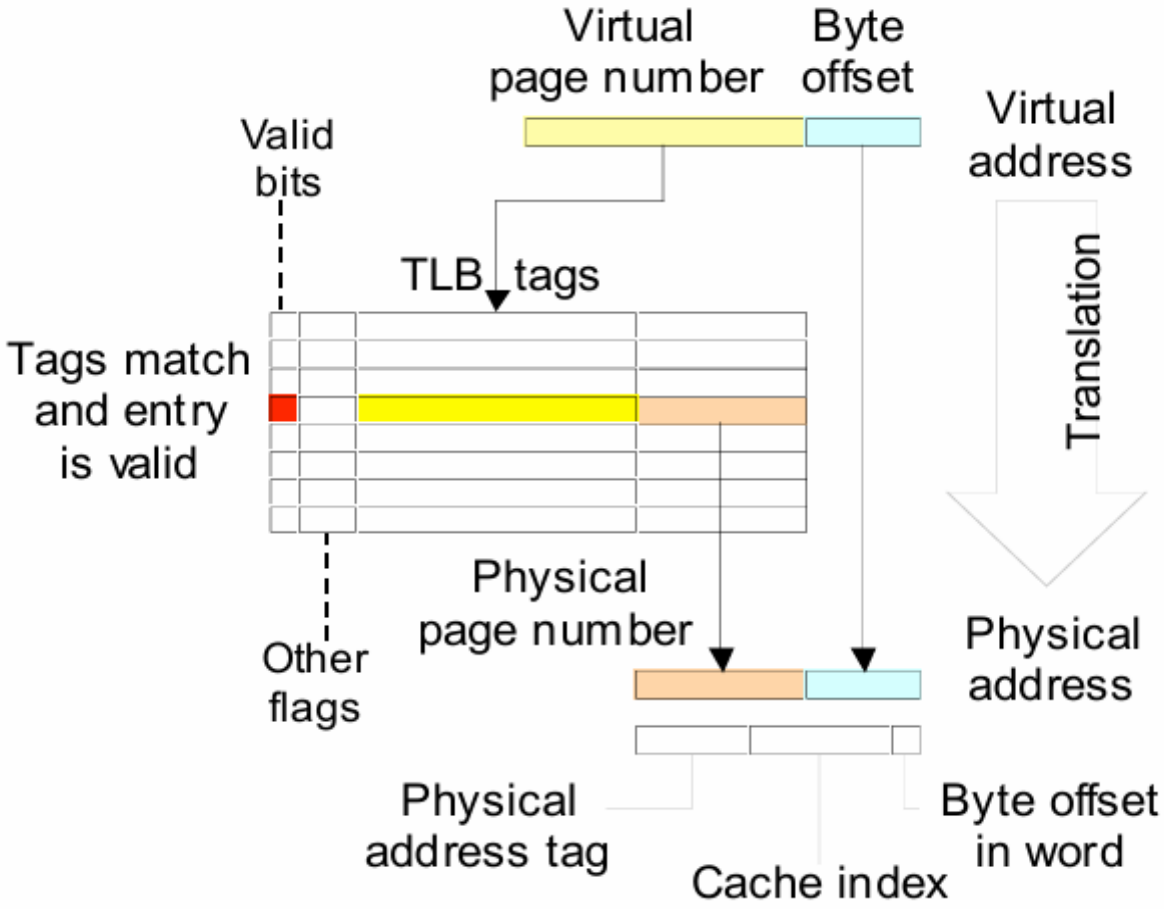
Department of Computer Science and Engineering
Yonghong Yan
yanyh@cse.sc.edu
https://passlab.github.io/CSCE513

Slides adapted from:
https://www.cs.nthu.edu.tw/~king/courses/cs5100/2014/N04-Cache-1.pdf

New Processor Architectures VM and Caches

Real Processor VM and Cache

Translation Lookaside Buffer

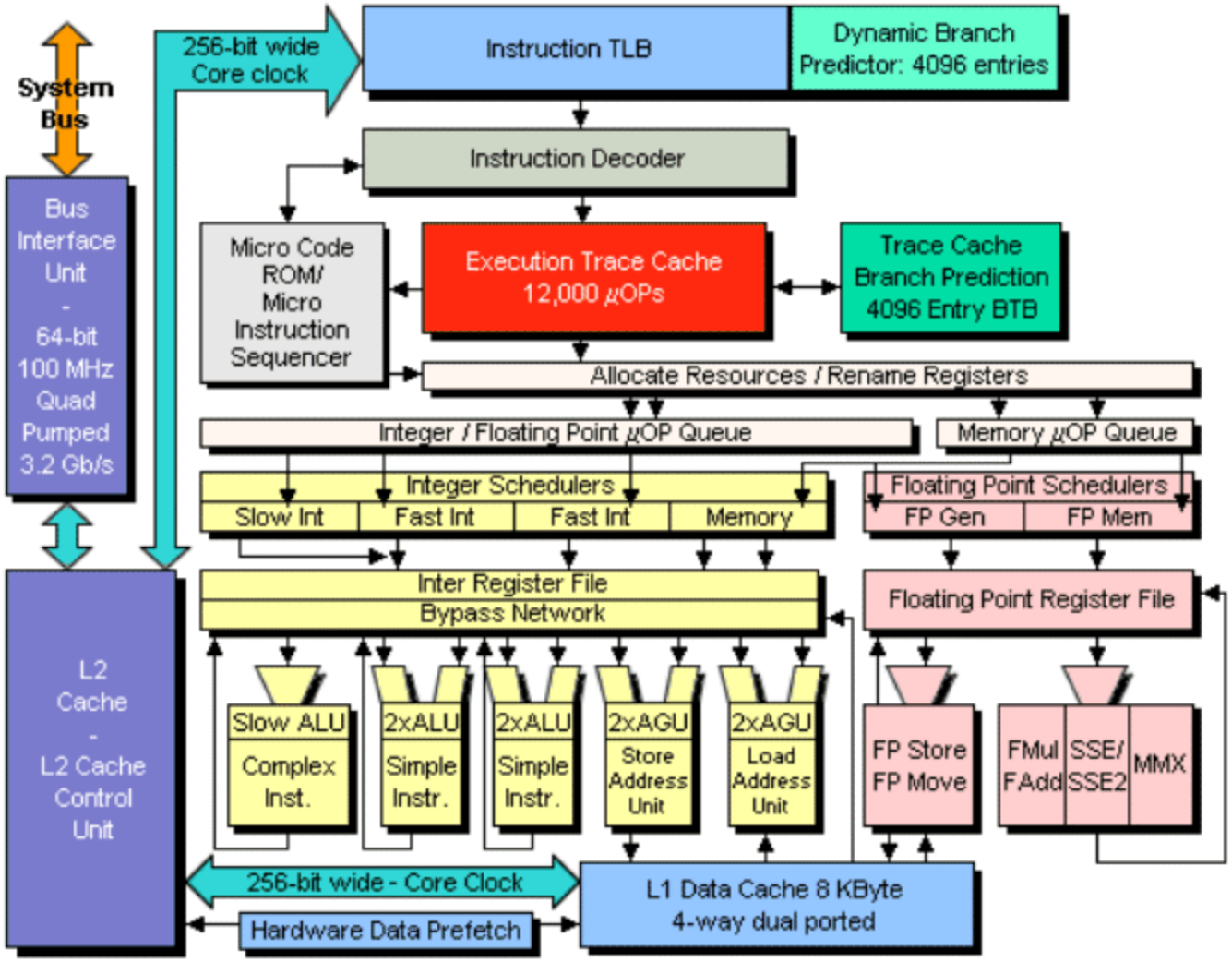


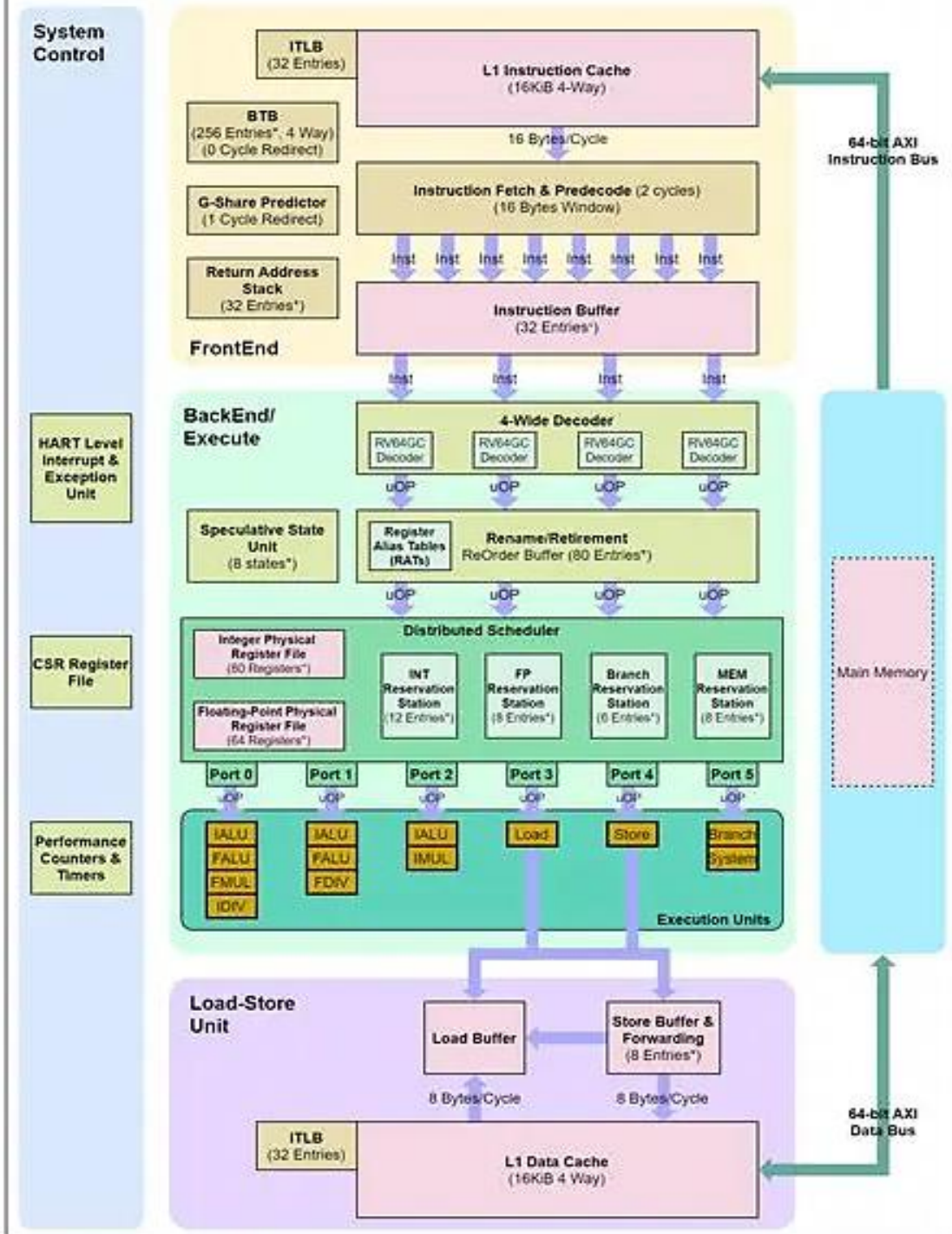
Virtual-to-physical address translation by a TLB and how the resulting physical address is used to access the cache memory.

Intel Pentium-Pro memory hierarchy

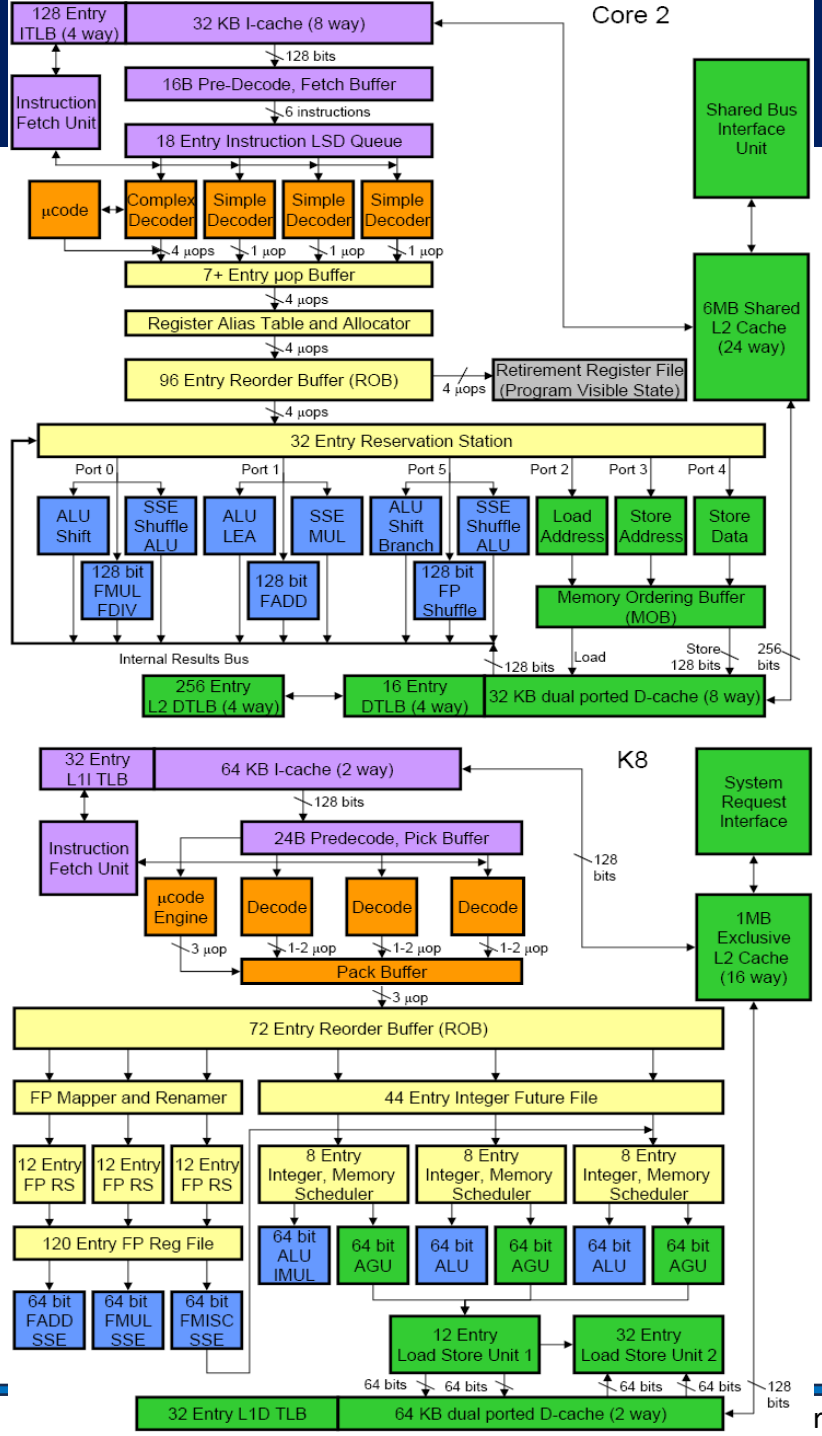
° Address Size:	32 bits (VA, PA)
° VM Page Size:	4 KB, 4 MB
° TLB organization:	separate i,d TLBs (i-TLB: 32 entries, d-TLB: 64 entries) 4-way set associative LRU approximated hardware handles miss
° L1 Cache:	8 KB, separate i,d 4-way set associative LRU approximated 32 byte block write back
° L2 Cache:	256 or 512 KB

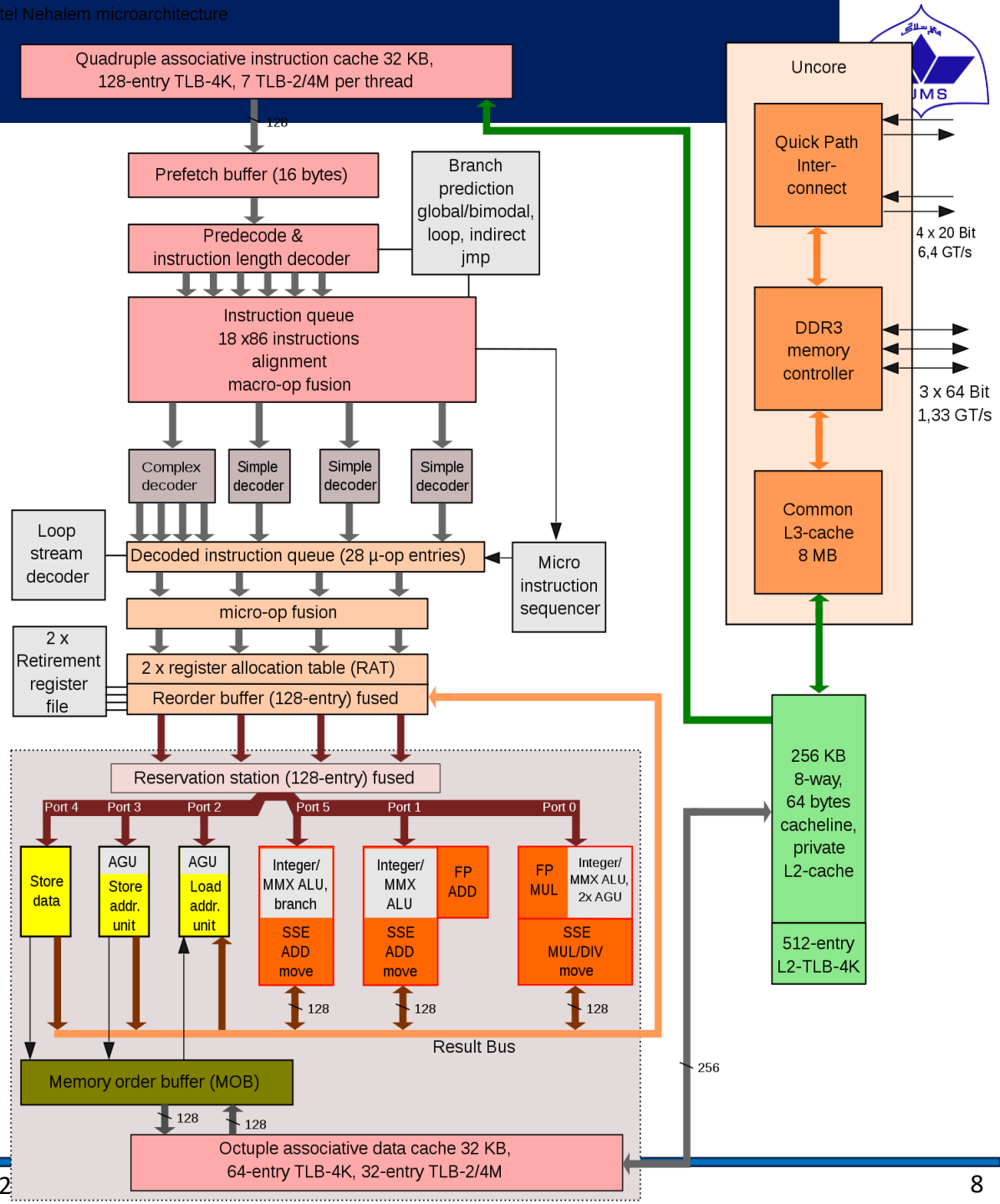
Some Processor Architectures



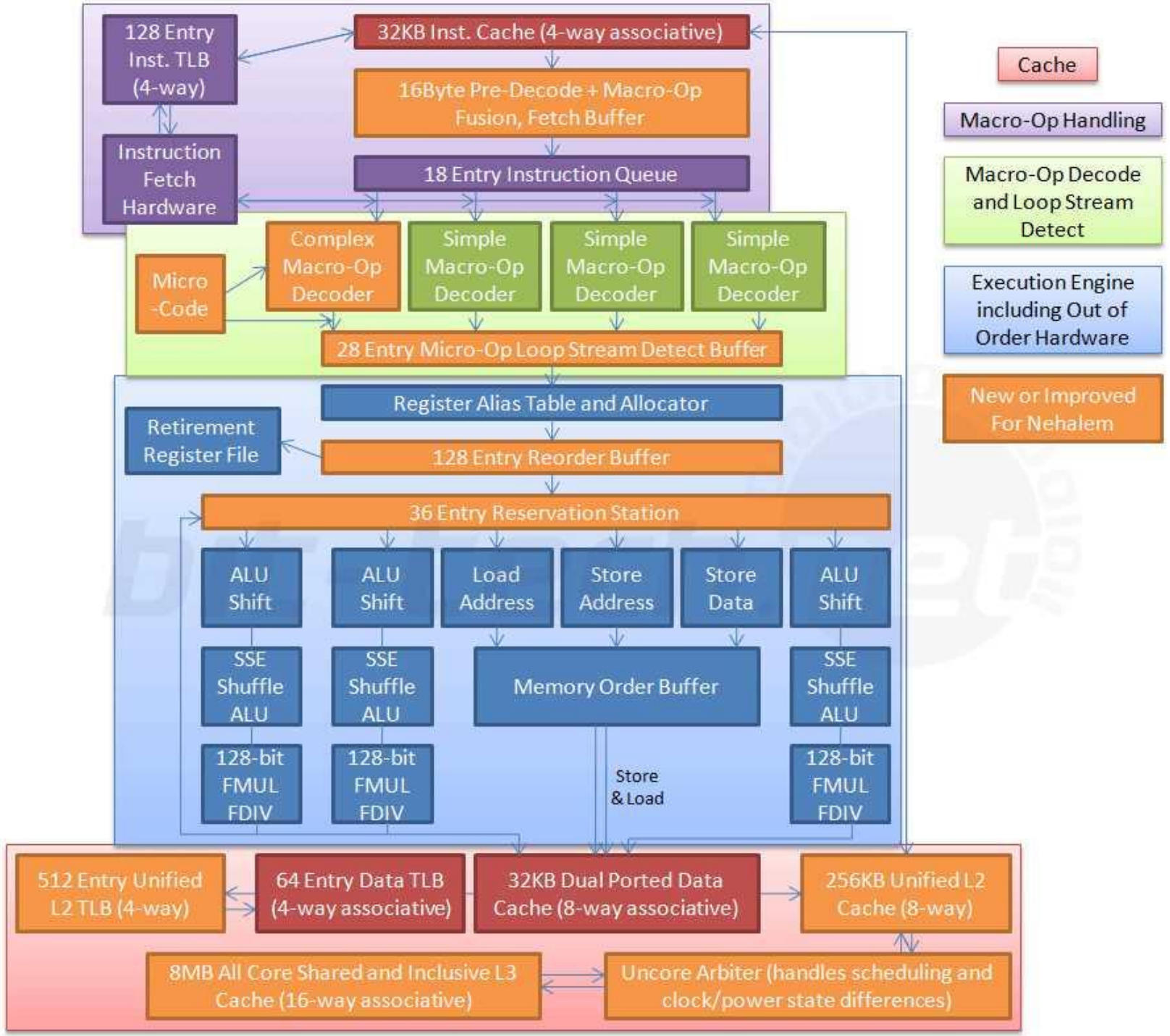


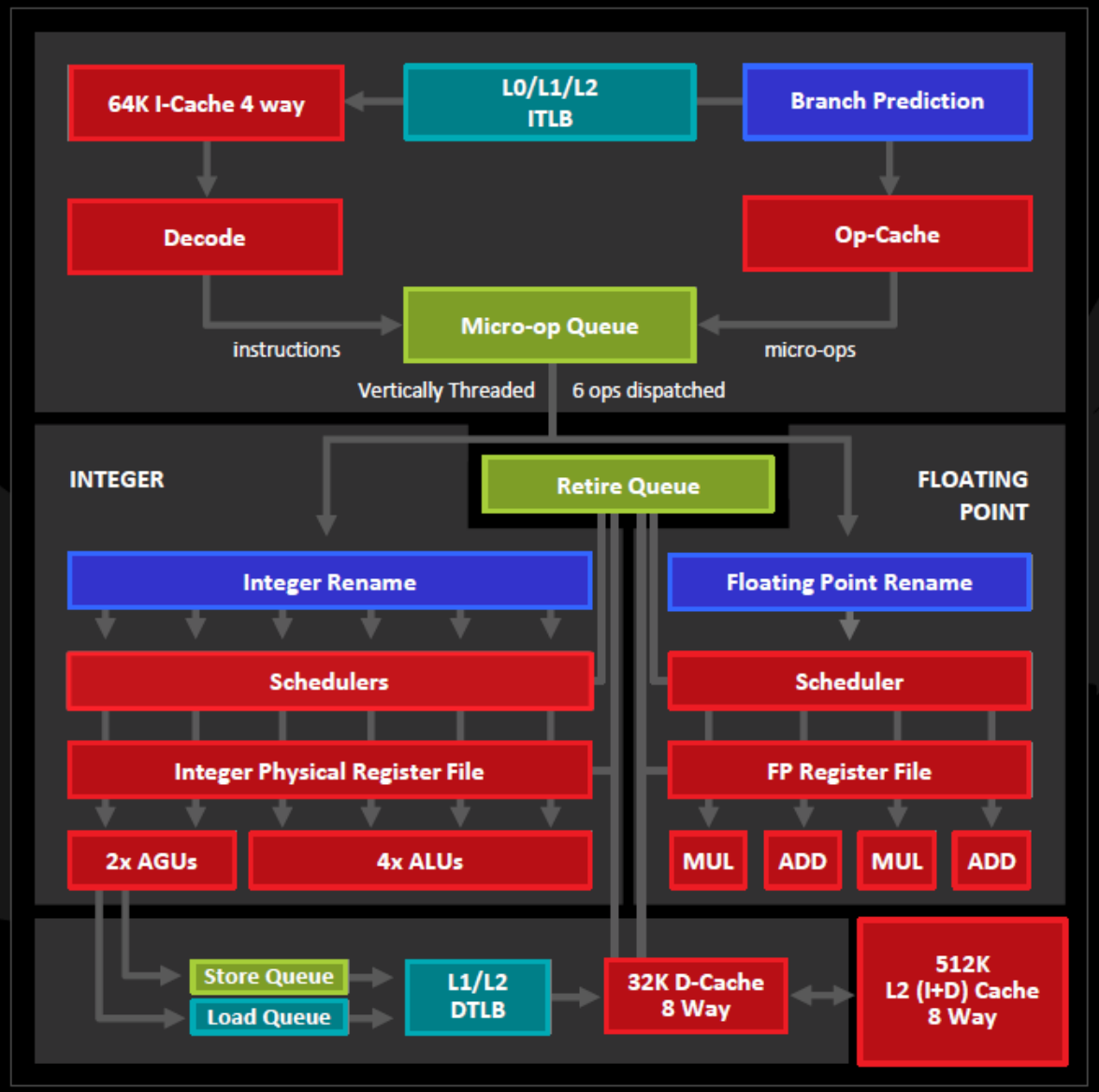
uProc Eg.


Core 2
128 Entry ITLB (4 way)
32 KB I-cache (8 way)
128 bits
16B Pre-Decode, Fetch Buffer
6 instructions
Instruction Fetch Unit
18 Entry Instruction LSD Queue
µcode
Complex Decoder
Simple Decoder
Simple Decoder
Simple Decoder
4 µops
1 µop
1 µop
1 µop
7+ Entry µop Buffer
4 µops
Register Alias Table and Allocator
4 µops
96 Entry Reorder Buffer (ROB)
4 µops
Retirement Register File (Program Visible State)
Shared Bus Interface Unit
6MB Shared L2 Cache (24 way)
4 µops
32 Entry Reservation Station
Port 0
Port 1
Port 5
Port 2
Port 3
Port 4
ALU Shift
SSE Shuffle ALU
ALU LEA
SSE MUL
ALU Shift Branch
SSE Shuffle ALU
Load Address
Store Address
Store Data
128 bit FMUL FDIV
128 bit FADD
128 bit FP Shuffle
Memory Ordering Buffer (MOB)
Internal Results Bus
128 bits
Load
Store 128 bits
256 bits
256 Entry L2 DTLB (4 way)
16 Entry DTLB (4 way)
32 KB dual ported D-cache (8 way)
K8
32 Entry L1I TLB
64 KB I-cache (2 way)
128 bits
System Request Interface
Instruction Fetch Unit
24B Predecode, Pick Buffer
128 bits
µcode Engine
Decode
Decode
Decode
3 µop
1-2 µop
1-2 µop
1-2 µop
Pack Buffer
1MB Exclusive L2 Cache (16 way)
3 µop
72 Entry Reorder Buffer (ROB)
FP Mapper and Renamer
44 Entry Integer Future File
12 Entry FP RS
12 Entry FP RS
12 Entry FP RS
8 Entry Integer, Memory Scheduler
8 Entry Integer, Memory Scheduler
8 Entry Integer, Memory Scheduler
120 Entry FP Reg File
64 bit ALU IMUL
64 bit AGU
64 bit ALU
64 bit AGU
64 bit ALU
64 bit AGU
64 bit FADD SSE
64 bit FMUL SSE
64 bit FMISC SSE
12 Entry Load Store Unit 1
32 Entry Load Store Unit 2
64 bits
64 bits
64 bits
64 bits
128 bits
32 Entry L1D TLB
64 KB dual ported D-cache (2 way)


Intel Nehalem microarchitecture
Quadruple associative instruction cache 32 KB, 128-entry TLB-4K, 7 TLB-2/4M per thread
128
Prefetch buffer (16 bytes)
Branch prediction global/bimodal, loop, indirect jmp
Predecode & instruction length decoder
Instruction queue 18 x86 instructions alignment macro-op fusion
Complex decoder
Simple decoder
Simple decoder
Simple decoder
Loop stream decoder
Decoded instruction queue (28 µ-op entries)
Micro instruction sequencer
micro-op fusion
2 x Retirement register file
2 x register allocation table (RAT)
Reorder buffer (128-entry) fused
Reservation station (128-entry) fused
Port 4
Port 3
Port 2
Port 5
Port 1
Port 0
Store data
AGU
Store addr. unit
AGU
Load addr. unit
Integer/ MMX ALU, branch
SSE ADD move
Integer/ MMX ALU
FP ADD
SSE ADD move
FP MUL
Integer/ MMX ALU, 2x AGU
SSE MUL/DIV move
128
128
128
Result Bus
Memory order buffer (MOB)
128
128
Octuple associative data cache 32 KB, 64-entry TLB-4K, 32-entry TLB-2/4M
Uncore
Quick Path Inter-connect
4 x 20 Bit 6,4 GT/s
DDR3 memory controller
3 x 64 Bit 1,33 GT/s
Common L3-cache 8 MB
256 KB 8-way, 64 bytes cacheline, private L2-cache
512-entry L2-TLB-4K
256

Intel Core i7





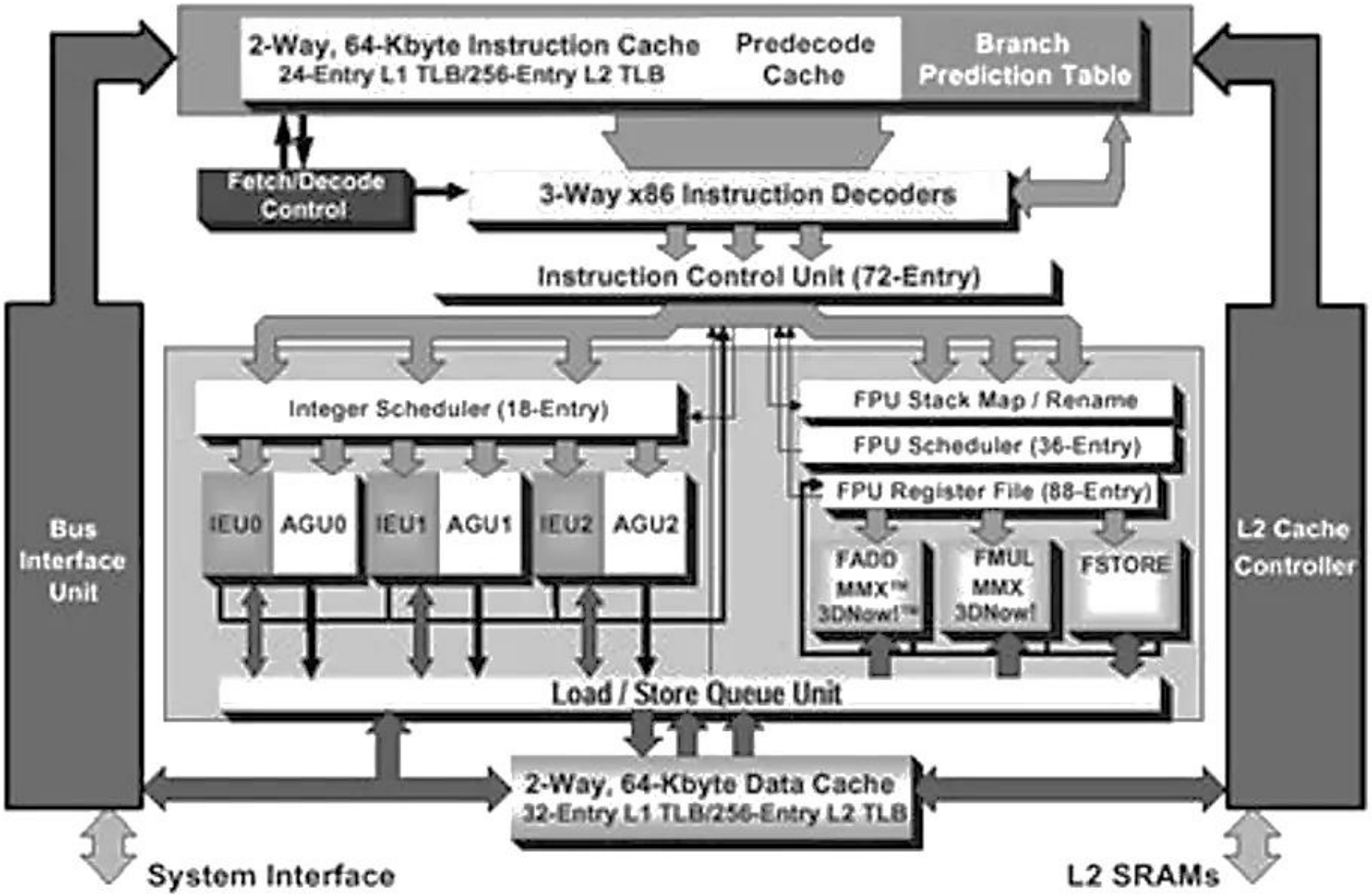
SMT OVERVIEW

- All structures fully available in 1T mode
- Front End Queues are round robin with priority overrides
- Increased throughput from SMT

- Competitively shared structures
- Competitively shared and SMT Tagged
- Competitively shared with Algorithmic Priority
- Statically Partitioned

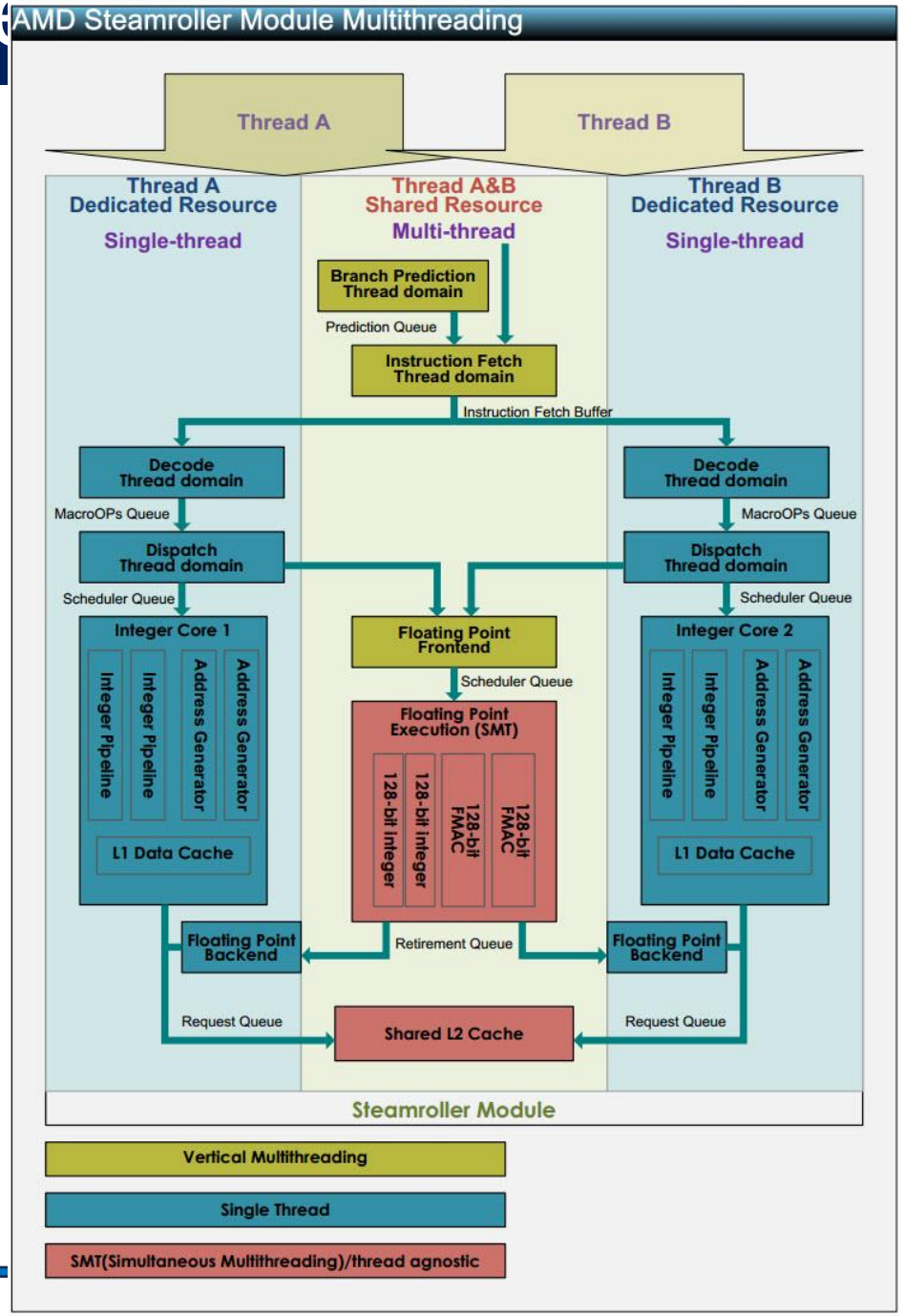
AMD Athlon

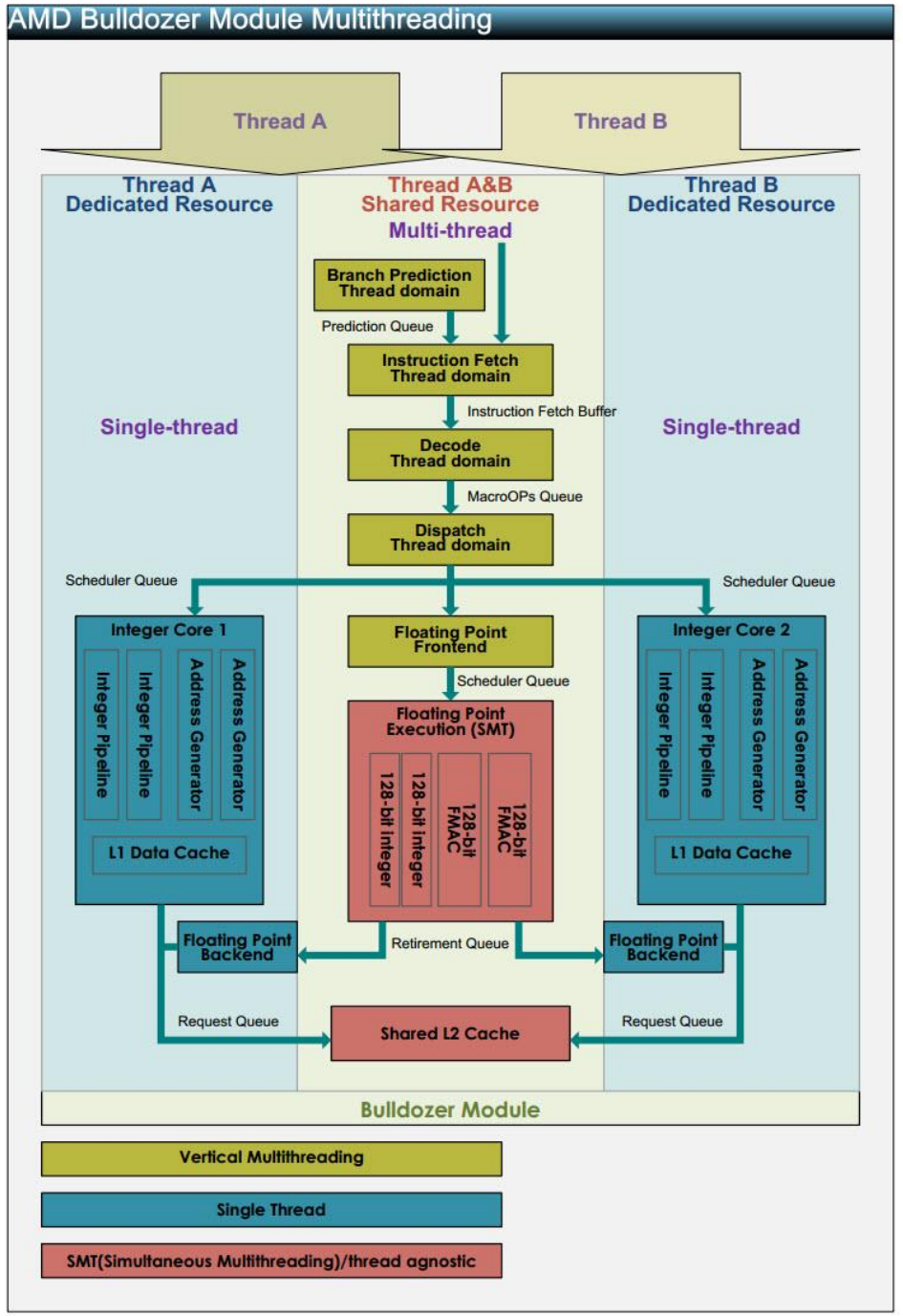
AMD Athlon K7



AMD Bulldoze

AMD Bulldozer Family Module Multithreading





CortexA9 Microarchitecture

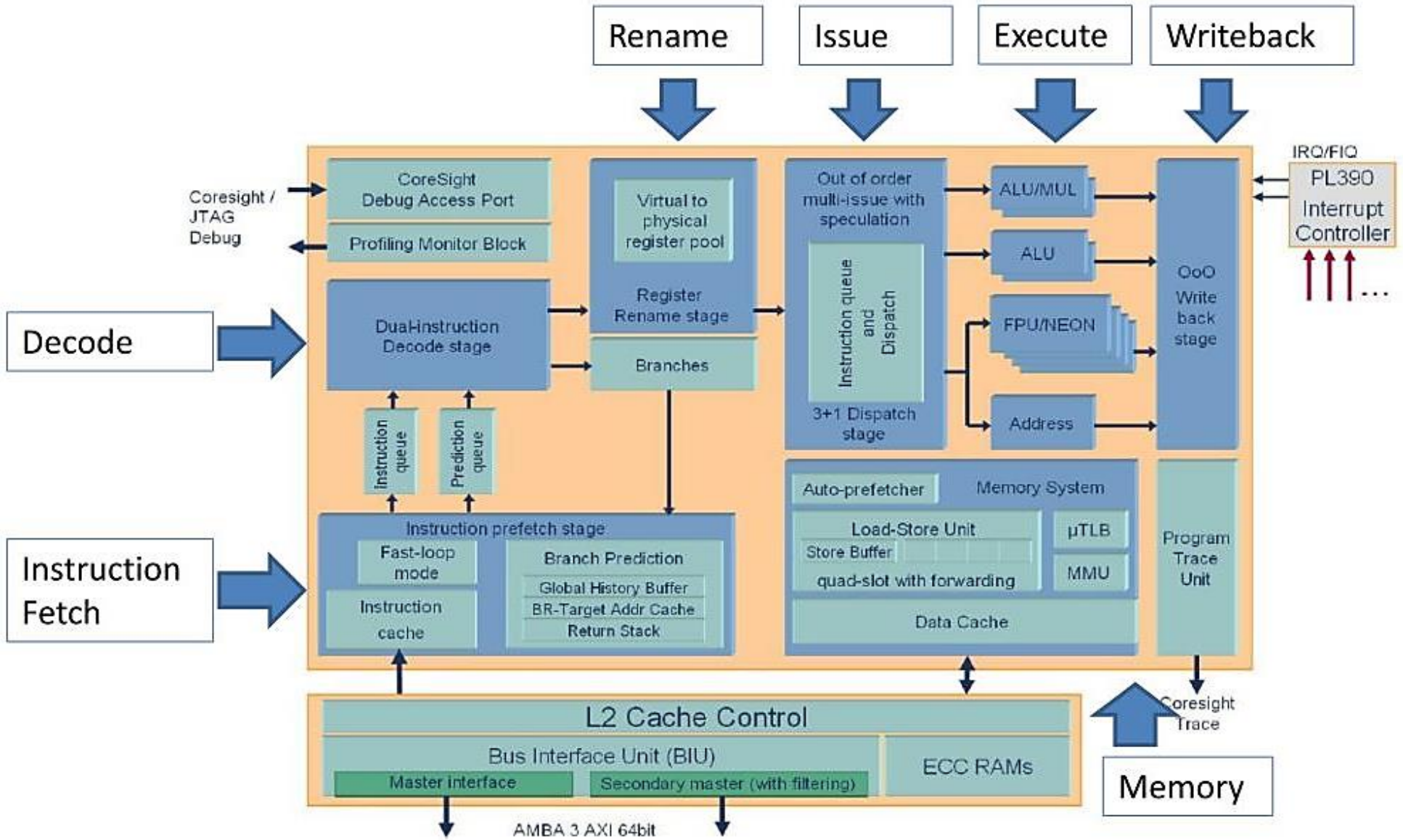


Fig. 1 Cortex-A9 microarchitecture structure and the single core interfaces.

www.arm.com/files/pdf/armcortexa-9processors.pdf

Page Fault, Page Table Size

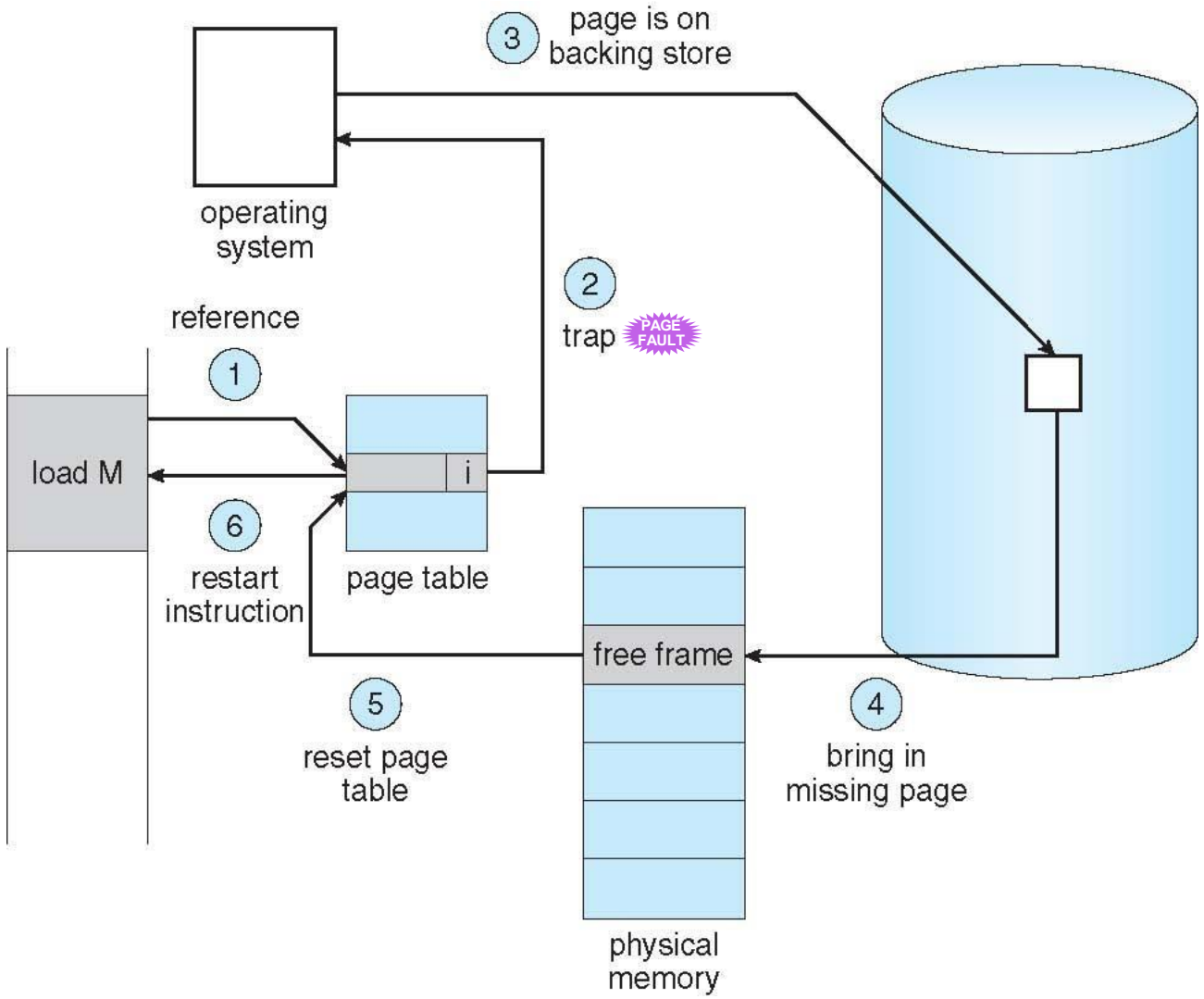
Handling Page Faults

- A page fault is like a cache miss
  - Must find page in lower level of hierarchy
- If valid bit is zero, the Physical Page Number points to a page on disk
- When OS starts new process, it creates space on disk for all the pages of the process, sets all valid bits in page table to zero, and all Physical Page Numbers to point to disk
  - called Demand Paging - pages of the process are loaded from disk only as needed

Page Fault

- Reference to a page with bit set to "i":
  Page fault
- OS looks at another table to decide:
  - Invalid reference? Abort.
- Just not in memory? Load it:
  - Get empty memory frame
  - Swap page into frame via scheduled disk operation
  - Reset tables to indicate page now in memory
    Set validation bit = "v"
  - Restart the instruction that caused the page fault

Steps in Handling a Page Fault - OS



Address Translation Mechanics

- The six questions
  - What? address translation
  - Why? compatibility, multi-programming, protection
  - How? page table
  - **Who performs it?**
  - **When?**
  - **Where does page table reside?**

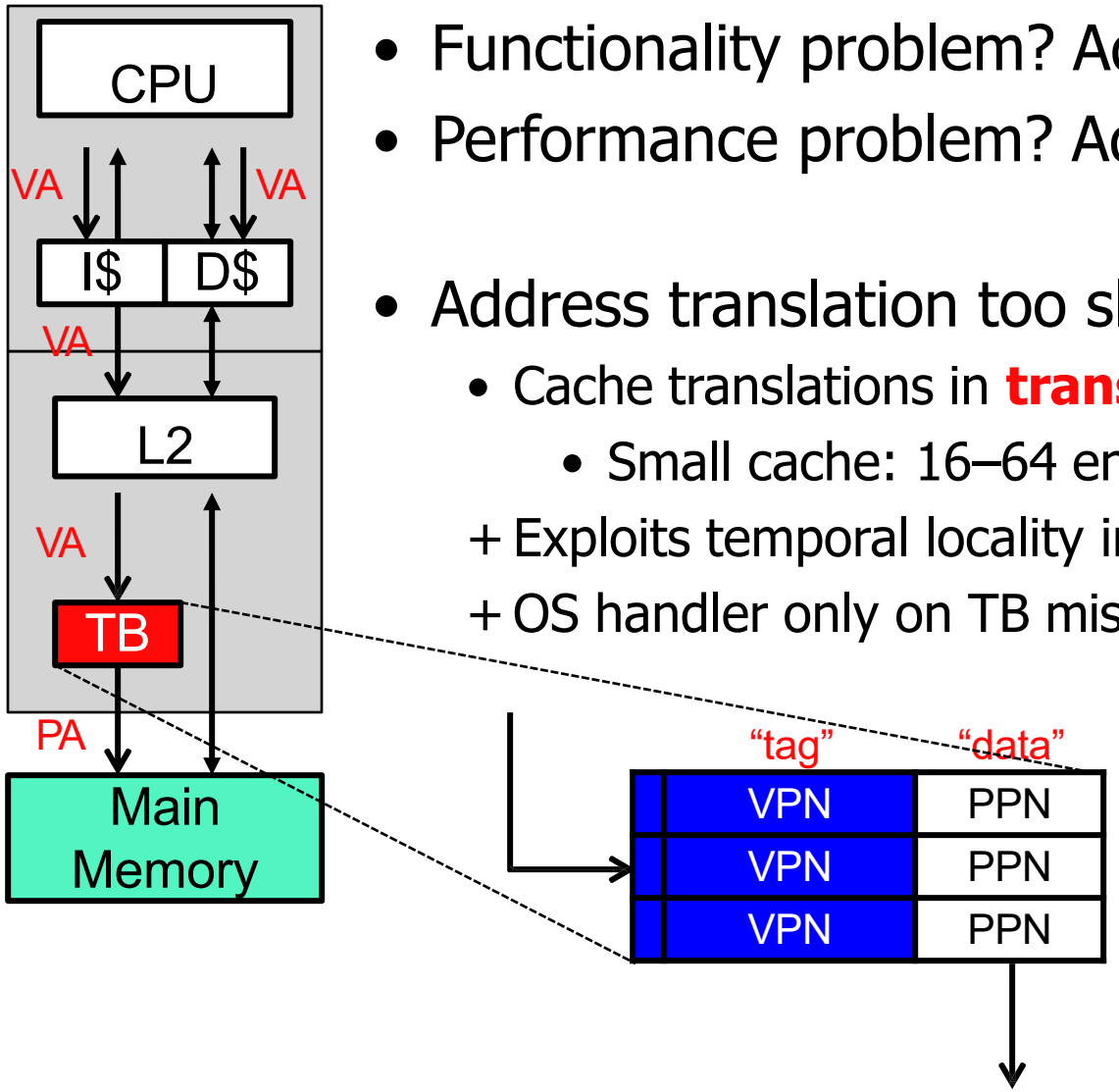
- Option I: process (program) translates its own addresses
  - Page table resides in process visible virtual address space
  - – Bad idea: implies that program (and programmer)…
    - …must know about physical addresses
      - Isn't that what virtual memory is designed to avoid?
    - …can forge physical addresses and mess with other programs
  - Translation on L2 miss or always? How would program know?

Who? Where? When? Take II

- Option II: **operating system (OS)** translates for process
  - Page table resides in OS virtual address space
  + User-level processes cannot view/modify their own tables
  + User-level processes need not know about physical addresses
  - Translation on L2 miss
    – Otherwise, OS SYSCALL before any fetch, load, or store

- L2 miss: interrupt transfers control to OS handler
  - Handler translates VA by accessing process's page table
  - Accesses memory using PA
  - Returns to user process when L2 fill completes
  – Still slow: added interrupt handler and PT lookup to memory access
  – What if PT lookup itself requires memory access? Head spinning…

Translation Buffer



- Functionality problem? Add indirection!
- Performance problem? Add cache!

- Address translation too slow?
  - Cache translations in **translation buffer (TB)**
    - Small cache: 16–64 entries, often fully assoc
  - \+ Exploits temporal locality in PT accesses
  - \+ OS handler only on TB miss

TLB Misses

- **TLB miss:** requested PTE not in TB, but in PT
  - Two ways of handling

- **1) OS routine**: reads PT, loads entry into TB (e.g., Alpha)
  - Privileged instructions in ISA for accessing TB directly
  - Latency: one or two memory accesses + OS call

- **2) Hardware FSM**: does same thing (e.g., IA-32)
  - Store PT root pointer in hardware register
  - Make PT root and 1st-level table pointers physical addresses
    - So FSM doesn't have to translate them

  \+ Latency: saves cost of OS call

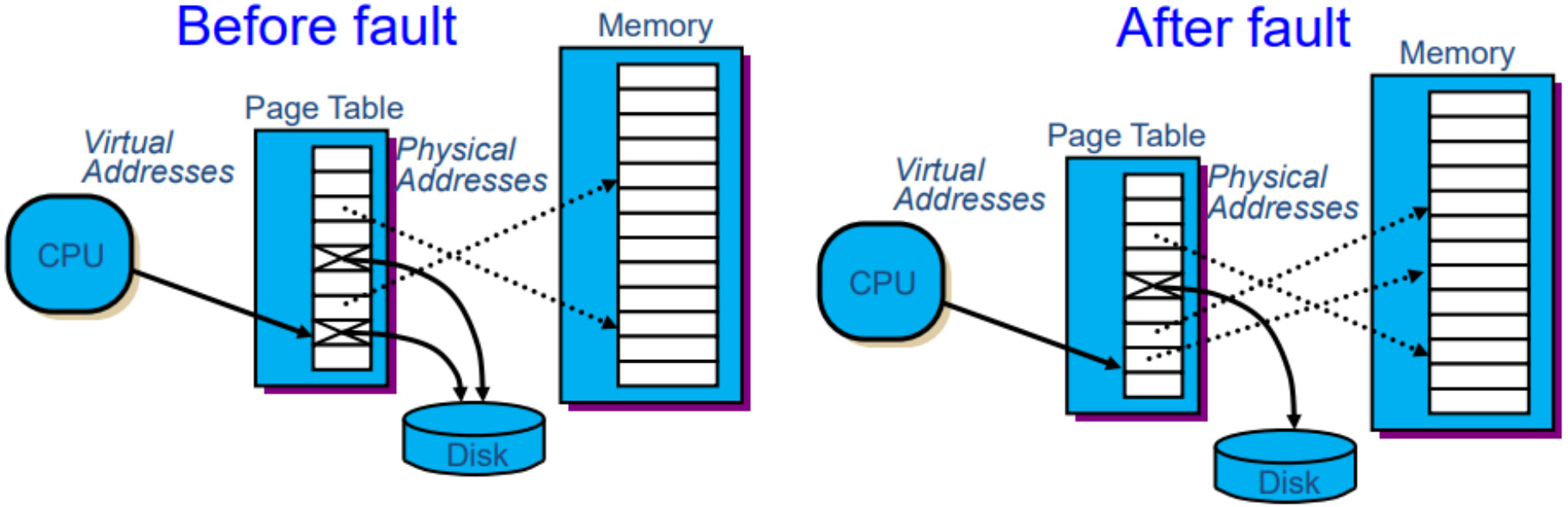
Page Faults

- **Page fault**: PTE not in TLB or in PT
  - Page is simply not in memory
  - Starts out as a TB miss, detected by OS handler/hardware FSM

- **OS routine**
  - OS software chooses a physical page to replace
    - **"Working set"**: more refined software version of LRU
      - Tries to see which pages are actively being used
      - Balances needs of all current running applications
    - If dirty, write to disk (like dirty cache block with writeback $)
  - Read missing page from disk (done by OS)
    - Takes so long (10ms), OS schedules another task
  - Treat like a normal TB miss from here

Handling Page Faults

- What if an object is on disk rather than in memory?
  - Page table entry indicates virtual address not in memory
  - OS exception handler invoked to move data from disk into memory
    - current process suspends, others can resume
    - OS has full control over placement, etc.



Selecting a Page Size

Selecting a Page Size

The most obvious architectural parameter is the page size. Choosing the page is a question of balancing forces that favor a larger page size versus those favoring a smaller size. The following favor a larger size:

- The size of the page table is inversely proportional to the page size; memory (or other resources used for the memory map) can therefore be saved by making the pages bigger.
- As mentioned in Section B.3, a larger page size can allow larger caches with fast cache hit times.
- Transferring larger pages to or from secondary storage, possibly over a network, is more efficient than transferring smaller pages.
- The number of TLB entries is restricted, so a larger page size means that more memory can be mapped efficiently, thereby reducing the number of TLB misses.

It is for this final reason that recent microprocessors have decided to support multiple page sizes; for some programs, TLB misses can be as significant on CPI as the cache misses.

Internal Fragmentation

The main motivation for a smaller page size is conserving storage. A small page size will result in less wasted storage when a contiguous region of virtual memory is not equal in size to a multiple of the page size. The term for this unused memory in a page is *internal fragmentation*. Assuming that each process has three primary segments (text, heap, and stack), the average wasted storage per process will be 1.5 times the page size. This amount is negligible for computers with hundreds of megabytes of memory and page sizes of 4–8 KiB. Of course, when the page sizes become very large (more than 32 KiB), storage (both main and secondary) could be wasted, as well as I/O bandwidth. A final concern is process start-up time; many processes are small, so a large page size would lengthen the time to invoke a process.

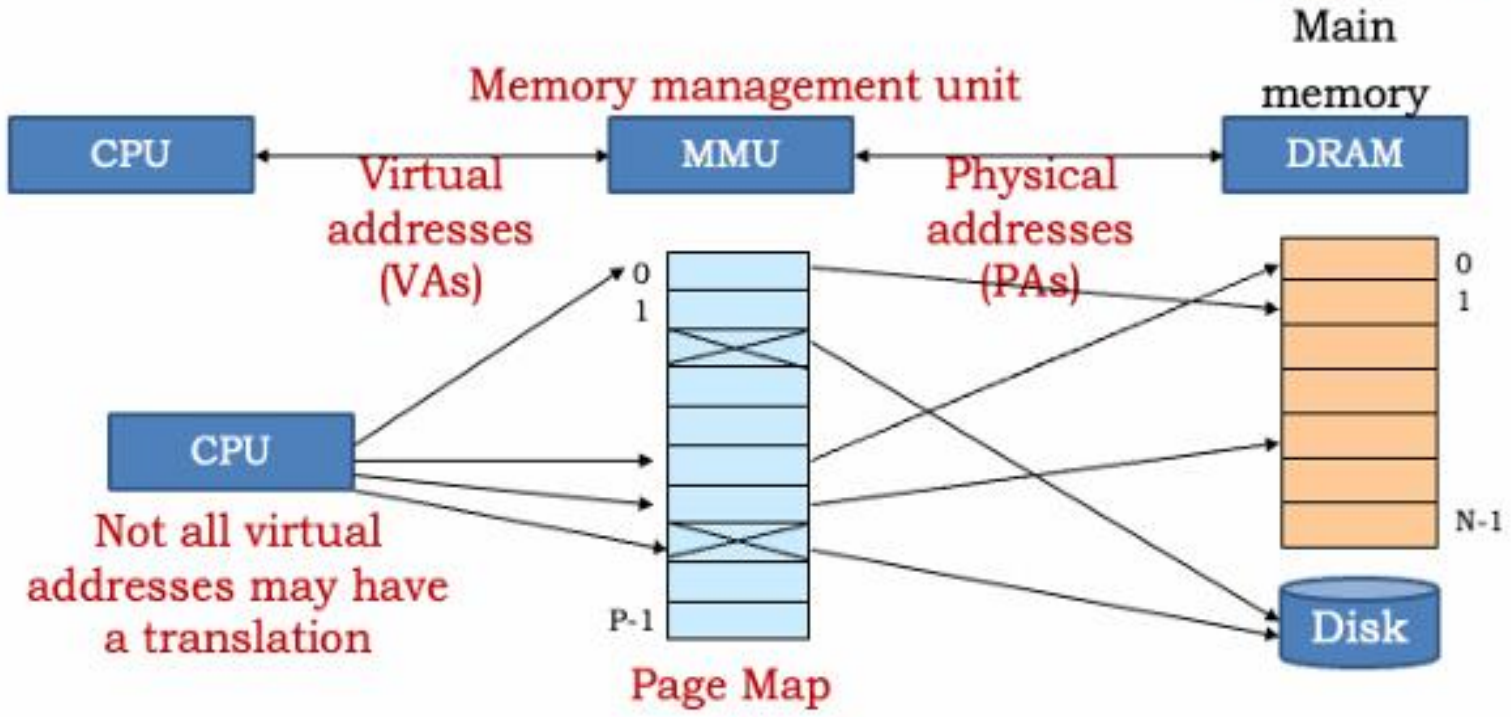
Another issue: Size of Page Tables

- Remember: For a 4 GiB address space, each process needs 4 GiB/4 KiB = 1 Million entries
  - Assuming 4 byte entries, 4 MiB per process
  - Per process! 256 concurrent processes consume 1 GiB just for the page table
  - “ps aux | wc -l” on my desktop tells me 261 processes are running…
- How do we reduce the size of page tables?
  - Larger page sizes? May waste memory capacity! (Fragmentation!)
  - Typical solution: Hierarchical page tables

Caches and Virtual Memory

Virtual Memory

- Two kinds of addresses:
  - CPU uses virtual addresses
  - Main memory uses physical addresses
- Hardware translates virtual addresses to physical addresses via an operating system (OS)-managed table, the page map



Where Can a Block be Placed in Main Memory?

Four Memory Hierarchy Questions Revisited

We are now ready to answer the four memory hierarchy questions for virtual memory.

Q1: Where Can a Block be Placed in Main Memory?

The miss penalty for virtual memory involves access to a rotating magnetic storage device and is therefore quite high. Given the choice of lower miss rates or a simpler placement algorithm, operating systems designers usually pick lower miss rates because of the exorbitant miss penalty. Thus, operating systems allow blocks to be placed anywhere in main memory. According to the terminology in Figure B.2 on page B-8, this strategy would be labeled fully associative.

How is a Block Found If it is in Main Memory?

Q2: How Is a Block Found If It Is in Main Memory?

Both paging and segmentation rely on a data structure that is indexed by the page or segment number. This data structure contains the physical address of the block. For segmentation, the offset is added to the segment's physical address to obtain the final physical address. For paging, the offset is simply concatenated to this physical page address (see Figure B.23).

This data structure, containing the physical page addresses, usually takes the form of a *page table*. Indexed by the virtual page number, the size of the table is the number of pages in the virtual address space. Given a 32-bit virtual address, 4 KiB pages, and 4 bytes per page table entry (PTE), the size of the page table would be $(2^{32}/2^{12}) \times 2^2 = 2^{22}$ or 4 MiB.

To reduce the size of this data structure, some computers apply a hashing function to the virtual address. The hash allows the data structure to be the length of the number of *physical* pages in main memory. This number could be much smaller than the number of virtual pages. Such a structure is called an *inverted page table*. Using the previous example, a 512 MiB physical memory would only need 1 MiB (8×512 MiB/4 KiB) for an inverted page table; the extra 4 bytes per page table entry are for the virtual address. The HP/Intel IA-64 covers both bases by offering both traditional pages tables *and* inverted page tables, leaving the choice of mechanism to the operating system programmer.

To reduce address translation time, computers use a cache dedicated to these address translations, called a *translation lookaside buffer*, or simply *translation buffer*, described in more detail shortly.

Which Block Should be Replaced on a Virtual Memory Miss?

Q3: Which Block Should be Replaced on a Virtual Memory Miss?

As mentioned earlier, the overriding operating system guideline is minimizing page faults. Consistent with this guideline, almost all operating systems try to replace the least recently used (LRU) block because if the past predicts the future, that is the one less likely to be needed.

To help the operating system estimate LRU, many processors provide a *use bit* or *reference bit*, which is logically set whenever a page is accessed. (To reduce work, it is actually set only on a translation buffer miss, which is described shortly.) The operating system periodically clears the use bits and later records them so it can determine which pages were touched during a particular time period. By keeping track in this way, the operating system can select a page that is among the least recently referenced.

What Happens on a Write in Virtual Memory?

Q4: What Happens on a Write?

The level below main memory contains rotating magnetic disks that take millions of clock cycles to access. Because of the great discrepancy in access time, no one has yet built a virtual memory operating system that writes through main memory to disk on every store by the processor. (This remark should not be interpreted as an opportunity to become famous by being the first to build one!) Thus, the write strategy is always write-back.

Because the cost of an unnecessary access to the next-lower level is so high, virtual memory systems usually include a dirty bit. It allows blocks to be written to disk only if they have been altered since being read from the disk.

Summary of Virtual Memory and Caches

Summary of Virtual Memory and Caches

With virtual memory, TLBs, first-level caches, and second-level caches all mapping portions of the virtual and physical address space, it can get confusing what bits go where. Figure B.25 gives a hypothetical example going from a 64-bit virtual address to a 41-bit physical address with two levels of cache. This L1 cache is virtually indexed, and physically tagged because both the cache size and the page size are 8 KiB. The L2 cache is 4 MiB. The block size for both is 64 bytes.

Imp

Memory Hierarchy from VA to L2 Cache

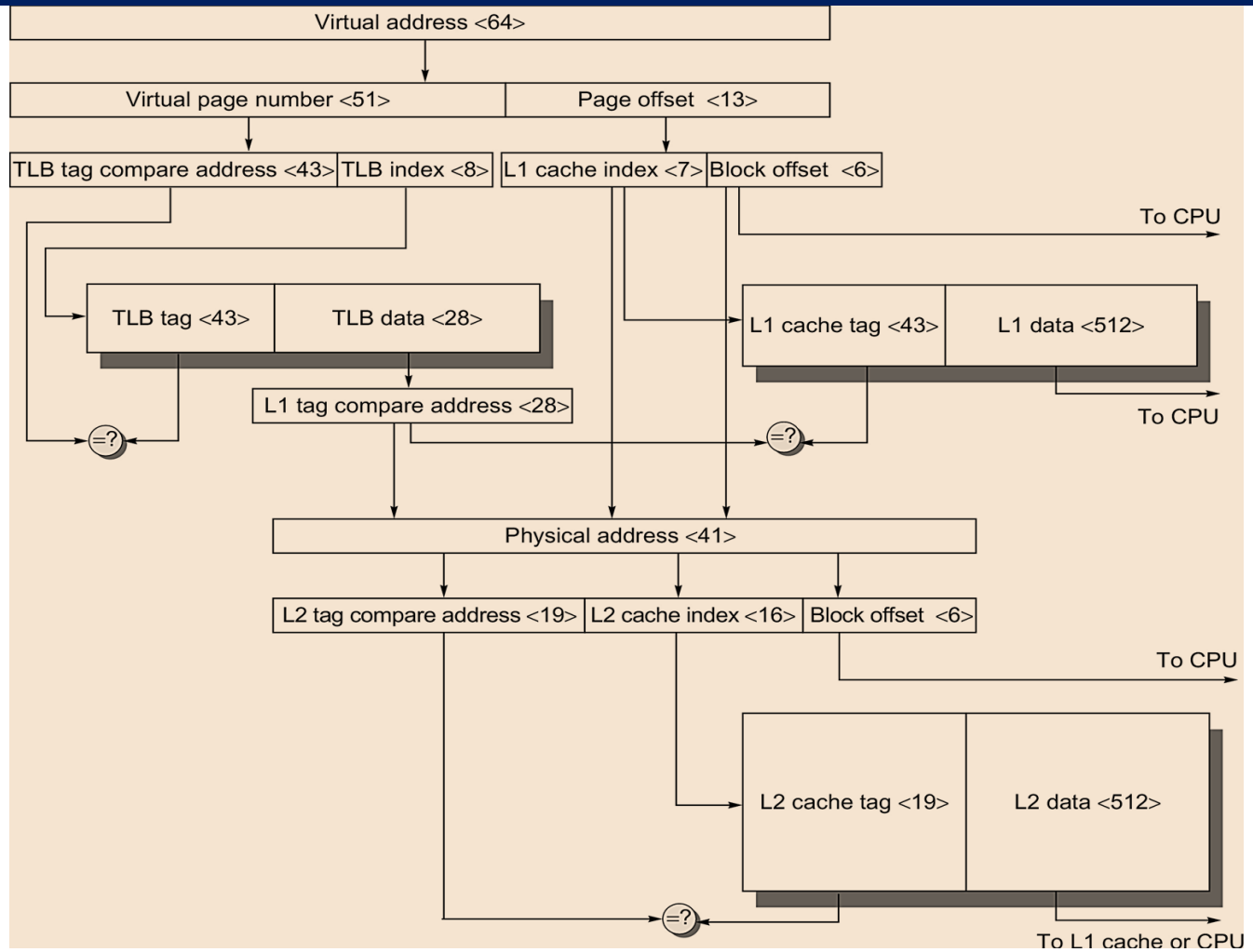


Figure B.25 The overall picture of a hypothetical memory hierarchy going from virtual address to L2 cache access. The page size is 8 KiB. The TLB is direct mapped with 256 entries. The L1 cache is a direct-mapped 8 KiB, and the L2 cache is a direct-mapped 4 MiB. Both use 64-byte blocks. The virtual address is 64 bits and the physical address is 41 bits. The primary difference between this simple figure and a real cache is replication of pieces of this figure.

Explanation VA to L2 Cache

First, the 64-bit virtual address is logically divided into a virtual page number and page offset. The former is sent to the TLB to be translated into a physical address, and the high bit of the latter is sent to the L1 cache to act as an index. If the TLB match is a hit, then the physical page number is sent to the L1 cache tag to check for a match. If it matches, it's an L1 cache hit. The block offset then selects the word for the processor.

If the L1 cache check results in a miss, the physical address is then used to try the L2 cache. The middle portion of the physical address is used as an index to the 4 MiB L2 cache. The resulting L2 cache tag is compared with the upper part of the physical address to check for a match. If it matches, we have an L2 cache hit, and the data are sent to the processor, which uses the block offset to select the desired word. On an L2 miss, the physical address is then used to get the block from memory.

Although this is a simple example, the major difference between this drawing and a real cache is replication. First, there is only one L1 cache. When there are two L1 caches, the top half of the diagram is duplicated. Note that this would lead to two TLBs, which is typical. Hence, one cache and TLB is for instructions, driven from the PC, and one cache and TLB is for data, driven from the effective address.

The second simplification is that all the caches and TLBs are direct mapped. If any were n-way set associative, then we would replicate each set of tag memory, comparators, and data memory n times and connect data memories with an n:1 multiplexor to select a hit. Of course, if the total cache size remained the same, the cache index would also shrink by $\log 2n$ bits according to the formula in Figure B.7 on page B-22.

Differences Between Cache and Virtual Memory

There are further differences between caches and virtual memory beyond those quantitative ones mentioned in Figure B.20:

- Replacement on cache misses is primarily controlled by hardware, while virtual memory replacement is primarily controlled by the operating system. The longer miss penalty means it's more important to make a good decision, so the operating system can be involved and take time deciding what to replace.
- The size of the processor address determines the size of virtual memory, but the cache size is independent of the processor address size.
- In addition to acting as the lower-level backing store for main memory in the hierarchy, secondary storage is also used for the file system. In fact, the file system occupies most of secondary storage. It is not usually in the address space.

Typical Parameters for VM and Caches

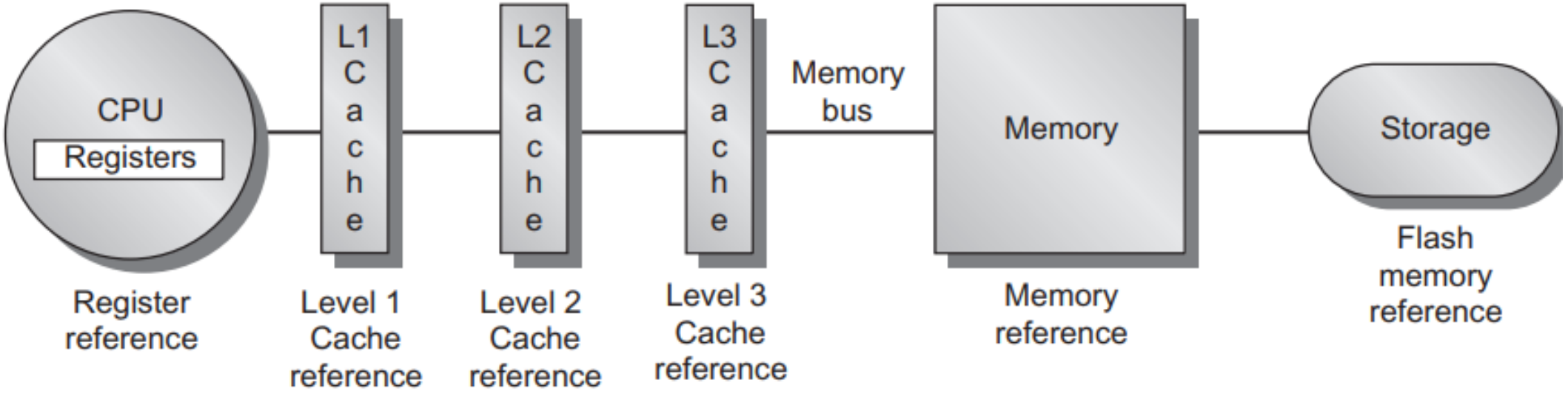
Parameter	First-level cache	Virtual memory
Block (page) size	16–128 bytes	4096–65,536 bytes
Hit time	1–3 clock cycles	100–200 clock cycles
Miss penalty	8–200 clock cycles	1,000,000–10,000,000 clock cycles
(access time)	(6–160 clock cycles)	(800,000–8,000,000 clock cycles)
(transfer time)	(2–40 clock cycles)	(200,000–2,000,000 clock cycles)
Miss rate	0.1%–10%	0.00001%–0.001%
Address mapping	25–45-bit physical address to 14–20-bit cache address	32–64-bit virtual address to 25–45-bit physical address

Figure B.20 Typical ranges of parameters for caches and virtual memory. Virtual memory parameters represent increases of 10–1,000,000 times over cache parameters. Usually, first-level caches contain at most 1 MiB of data, whereas physical memory contains 256 MiB to 1 TB.

- https://www.intel.com/content/www/us/en/docs/programmable/683567/24-3/system-mmu-block-diagram-falconmesa.html

Memory Hierarchy, Controller and Memory Technology

Simple Memory Hierarchy



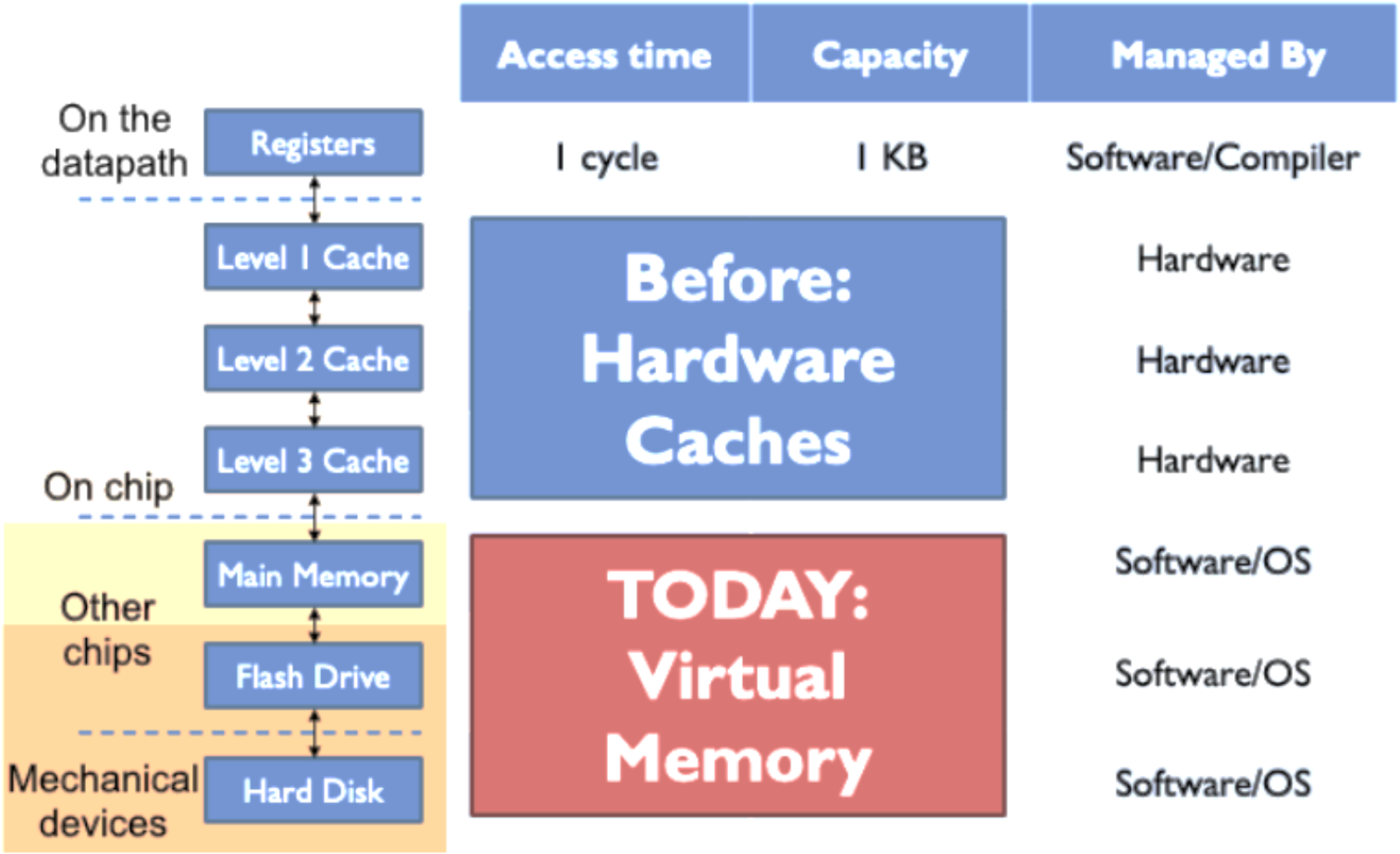
Laptop	Size:	1000 bytes	64 KB	256 KB	4-8 MB	4–16 GB	256 GB-1 TB
	Speed:	300 ps	1 ns	3–10 ns	10–20 ns	50–100 ns	50-100 uS
Desktop	Size:	2000 bytes	64 KB	256 KB	8-32 MB	8–64 GB	256 GB-2 TB
	Speed:	300 ps	1 ns	3–10 ns	10–20 ns	50–100 ns	50-100 uS

(B) Memory hierarchy for a laptop or a desktop

Memory Hierarchy with Virtual Memory

Reminder: A Typical Memory Hierarchy

- Everything is a cache for something else

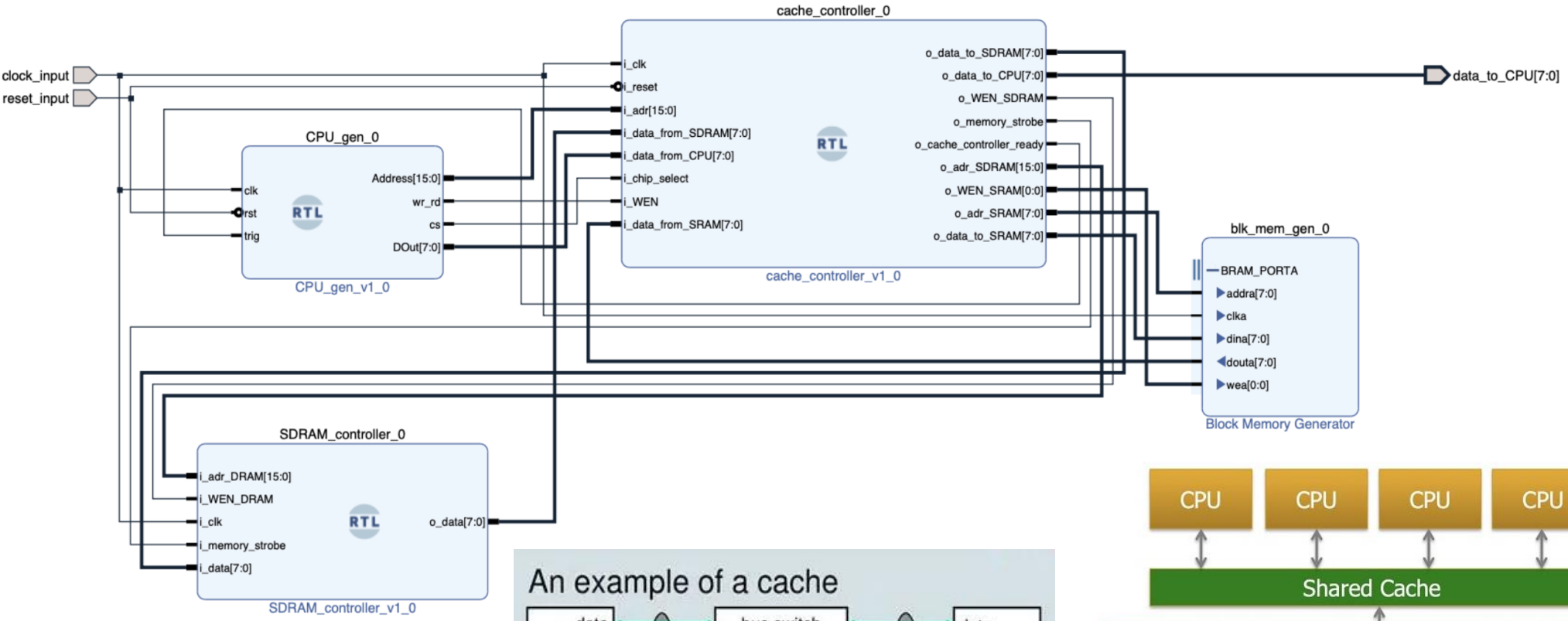


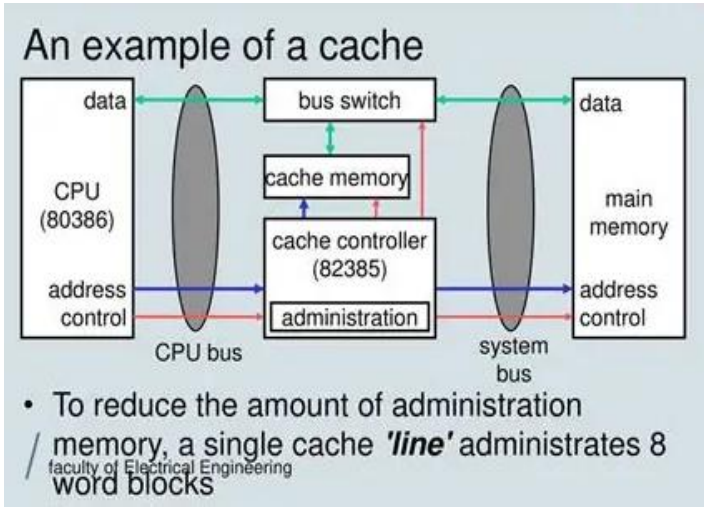
Access time	Capacity	Managed By
1 cycle	1 KB	Software/Compiler
		Hardware
		Hardware
		Hardware
		Software/OS
		Software/OS
		Software/OS

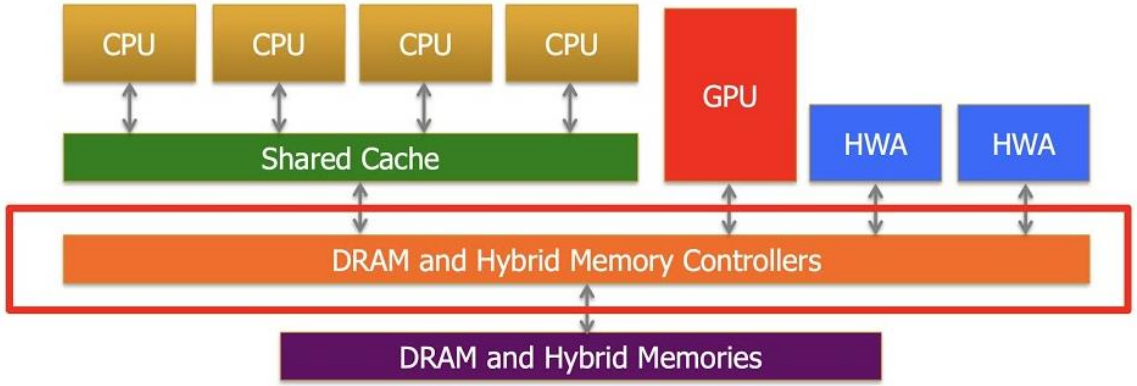
Before: Hardware Caches

TODAY: Virtual Memory

Cache Controller and Memory Controller



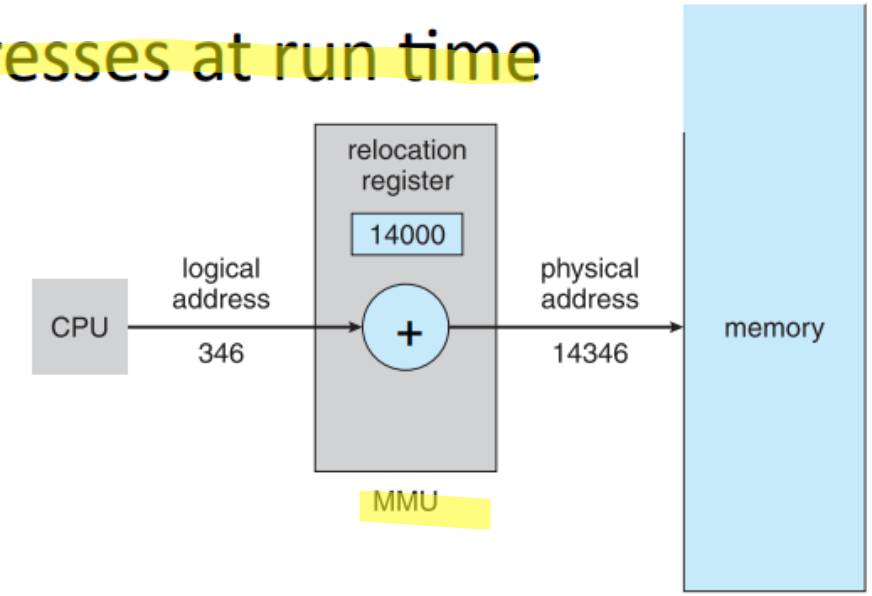




- Heterogeneous agents: CPUs, GPUs, and HWAs
- Main memory interference between CPUs, GPUs, HWAs
- Many timing constraints for various memory types
- Many goals at the same time: performance, fairness, QoS, energy efficiency, …

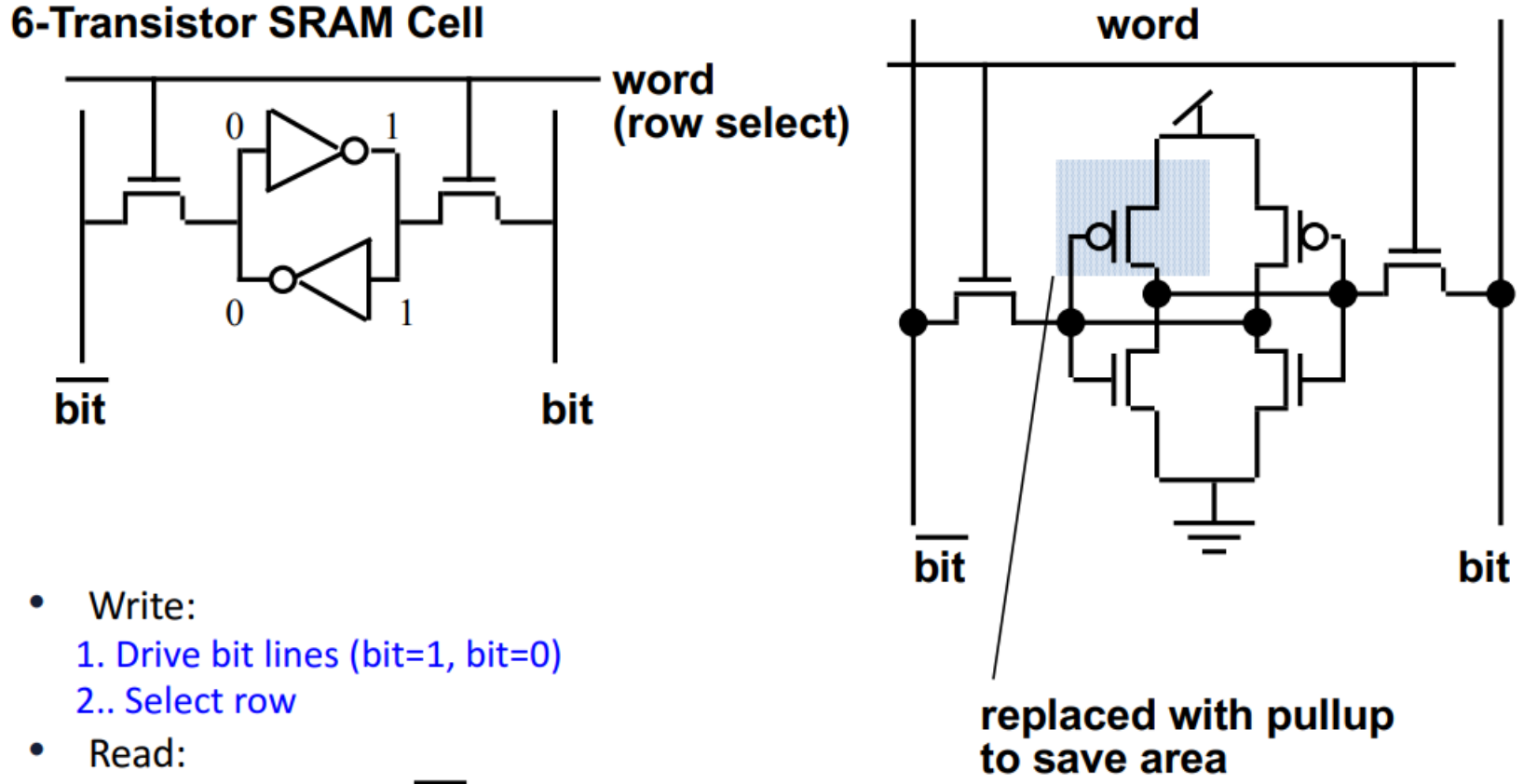
Memory Management Unit (MMU)

- Hardware that maps logical to physical addresses at run time
- Conceptually simple scheme: replace base register with **relocation register**
- Add the value in the relocation register to every address generated by a user process at the time it is sent to memory
  - User programs deal with logical addresses, never seeing physical addresses
- Execution-time binding occurs when reference is made to location in memory
  - Logical address is bound to physical address by the MMU



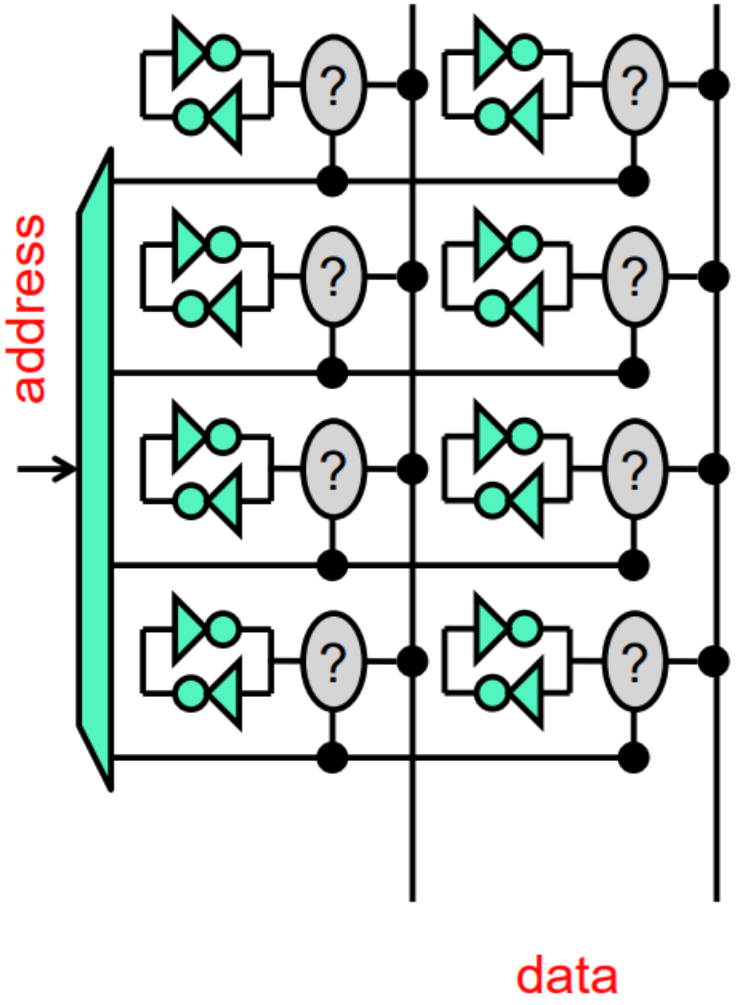
Static RAM Cell Design

Static RAM (SRAM) 6-Transistor Cell – 1 Bit



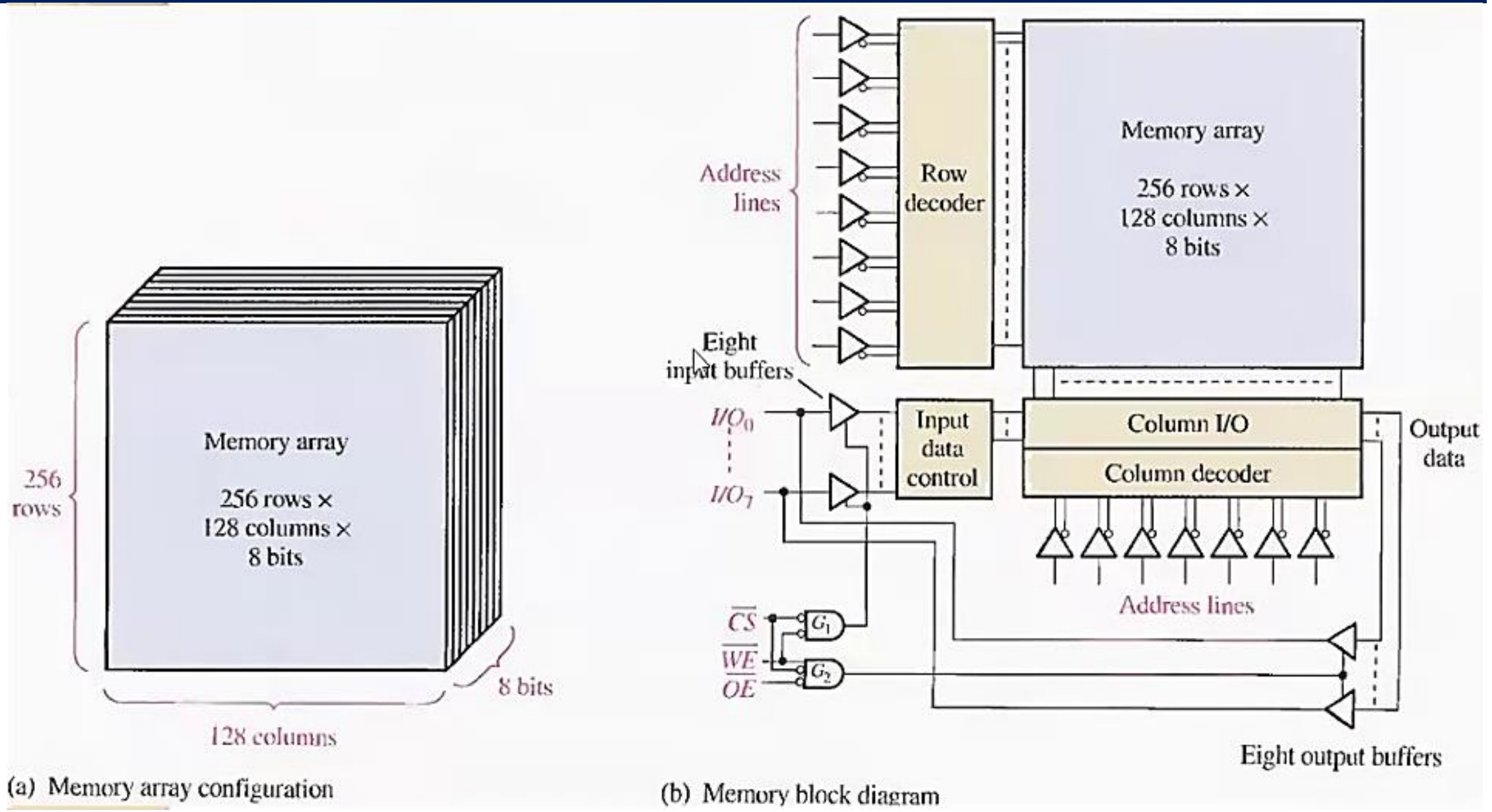
- Write:
  1. Drive bit lines (bit=1, bit=0)
  2.. Select row
- Read:
  1. Precharge bit and $\overline{bit}$ to Vdd or Vdd/2 => make sure equal!
  2.. Select row
  3. Cell pulls one line low
  4. Sense amp on column detects difference between bit and $\overline{bit}$

Static RAM (SRAM)



- **SRAM**: static RAM
  - Bits as cross-coupled inverters
  - Four transistors per bit
  - More transistors for ports
- **"Static"** means
  - Inverters connected to power/ground
  - Bits naturally/continuously "refreshed"
  - Bit values never decay
- Designed for speed

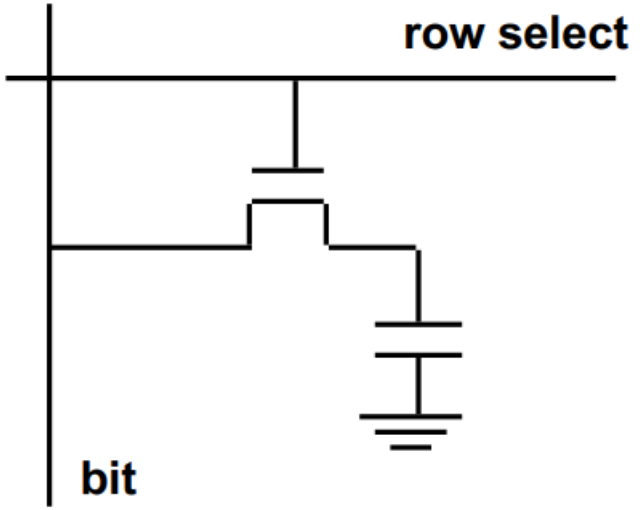
SRAM Organization



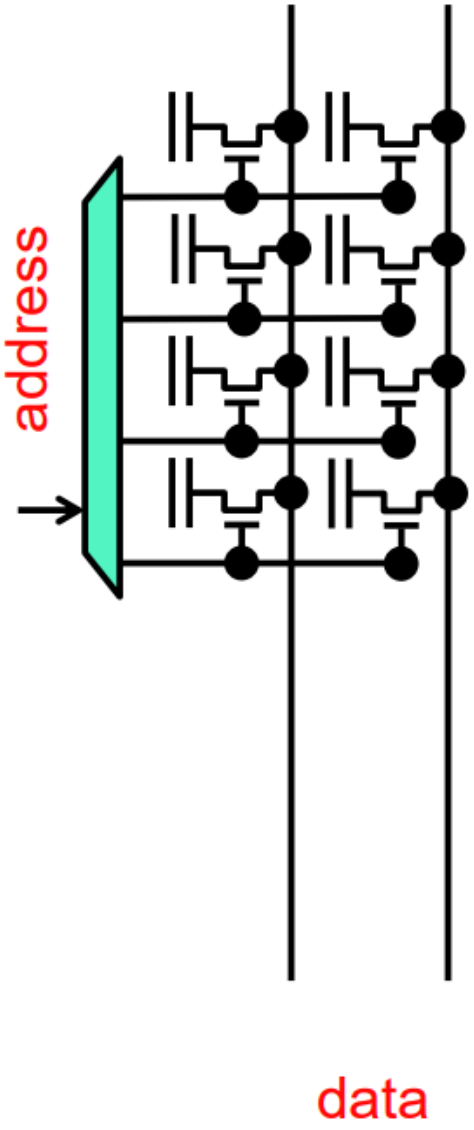
DRAM Cell Design

Dynamic RAM (DRAM) 1-Transistor Memory Cell

- Write:
  - 1. Drive bit line
  - 2. Select row
- Read:
  - 1. Precharge bit line to Vdd
  - 2. Select row
  - 3. Cell and bit line share charges
    - Very small voltage changes on the bit line
  - 4. Sense (fancy sense amp)
    - Can detect changes of ~1 million electrons
  - 5. Write: restore the value
- Refresh
  - 1. Just do a dummy read to every cell.



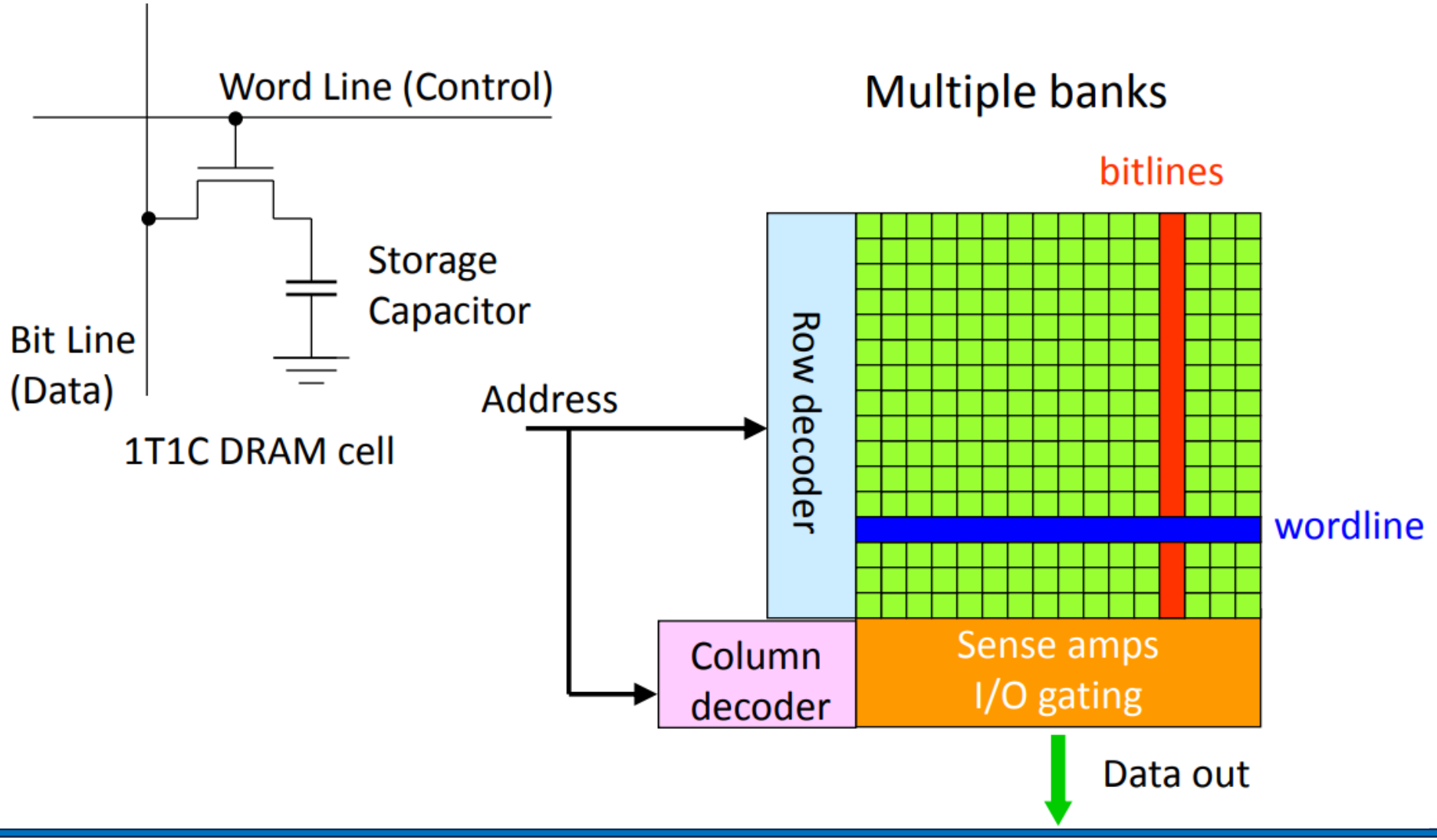
Dynamic RAM (DRAM)



- **DRAM**: dynamic RAM
  - Bits as capacitors (if charge, bit=1)
  - "Pass transistors" as ports
  - One transistor per bit/port
- **"Dynamic"** means
  - Capacitors not connected to power/gnd
  - Stored charge decays over time
  - Must be explicitly refreshed
- Designed for density
  - Moore's Law …

DRAM Bank

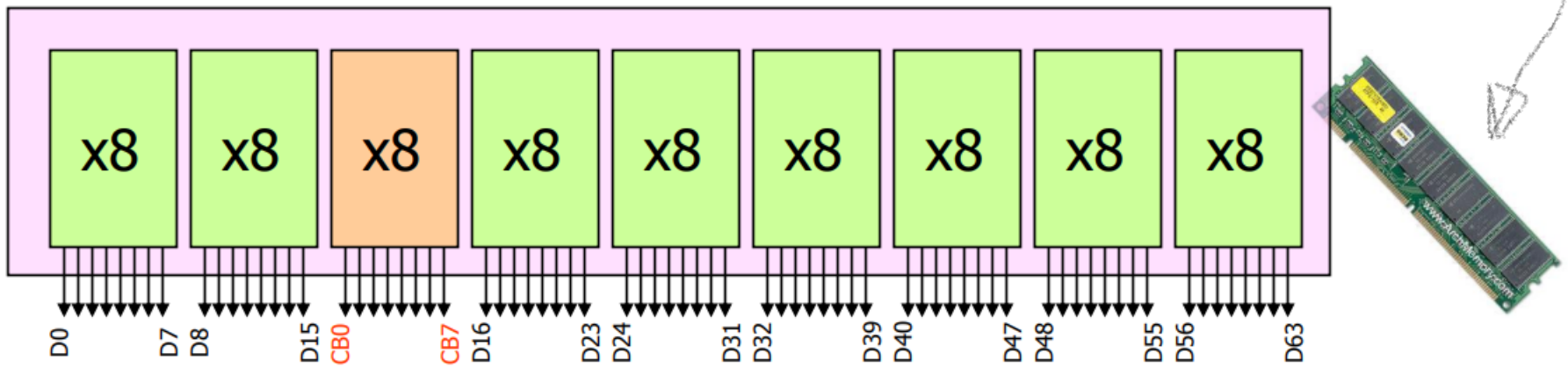
DRAM Cell and DRAM Bank



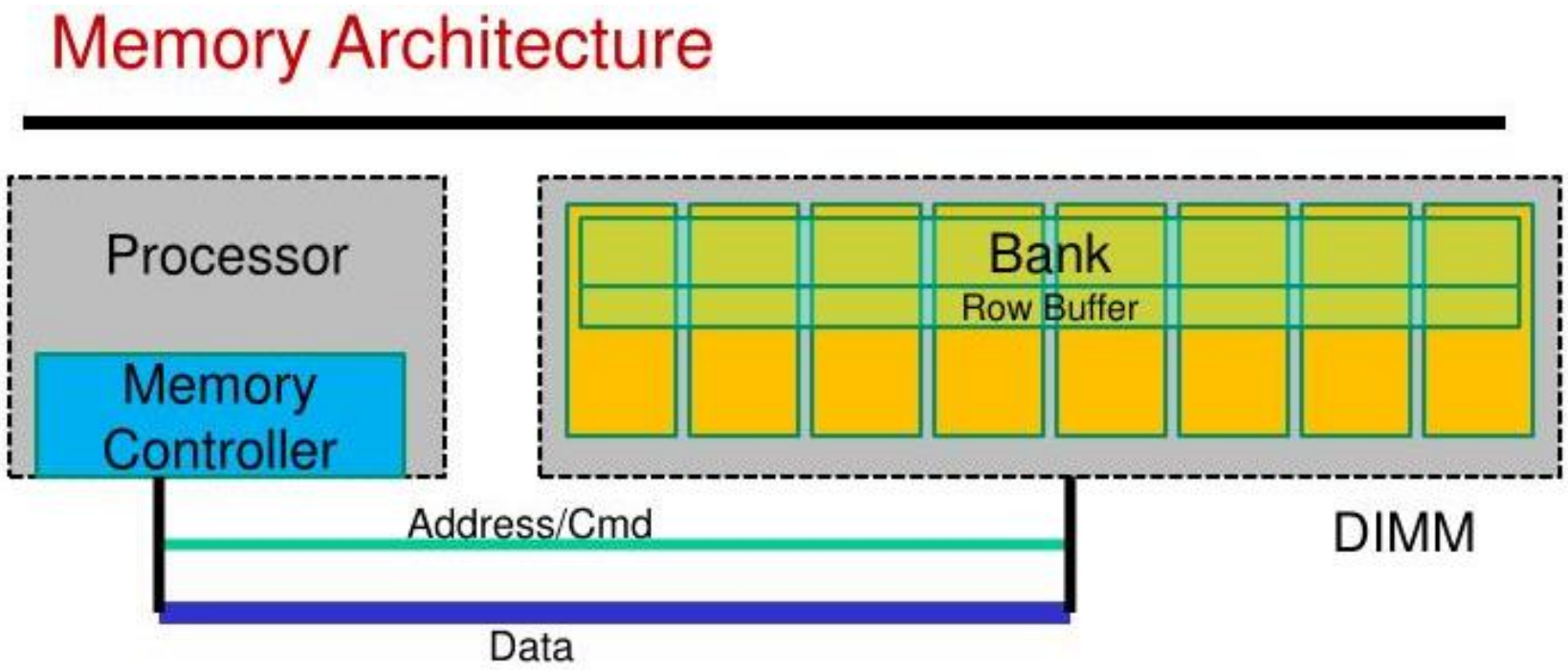
DRAM Organization

- Memory bank
  - Different banks can operate independently
- DRAM rank
  - Chips inside the same rank are accessed simultaneously
- DIMM (Dual Inline Memory Module)
  - Typically contains 4 to 16 DRAM chips organized to be 8 bytes wide

X72 (ECC)

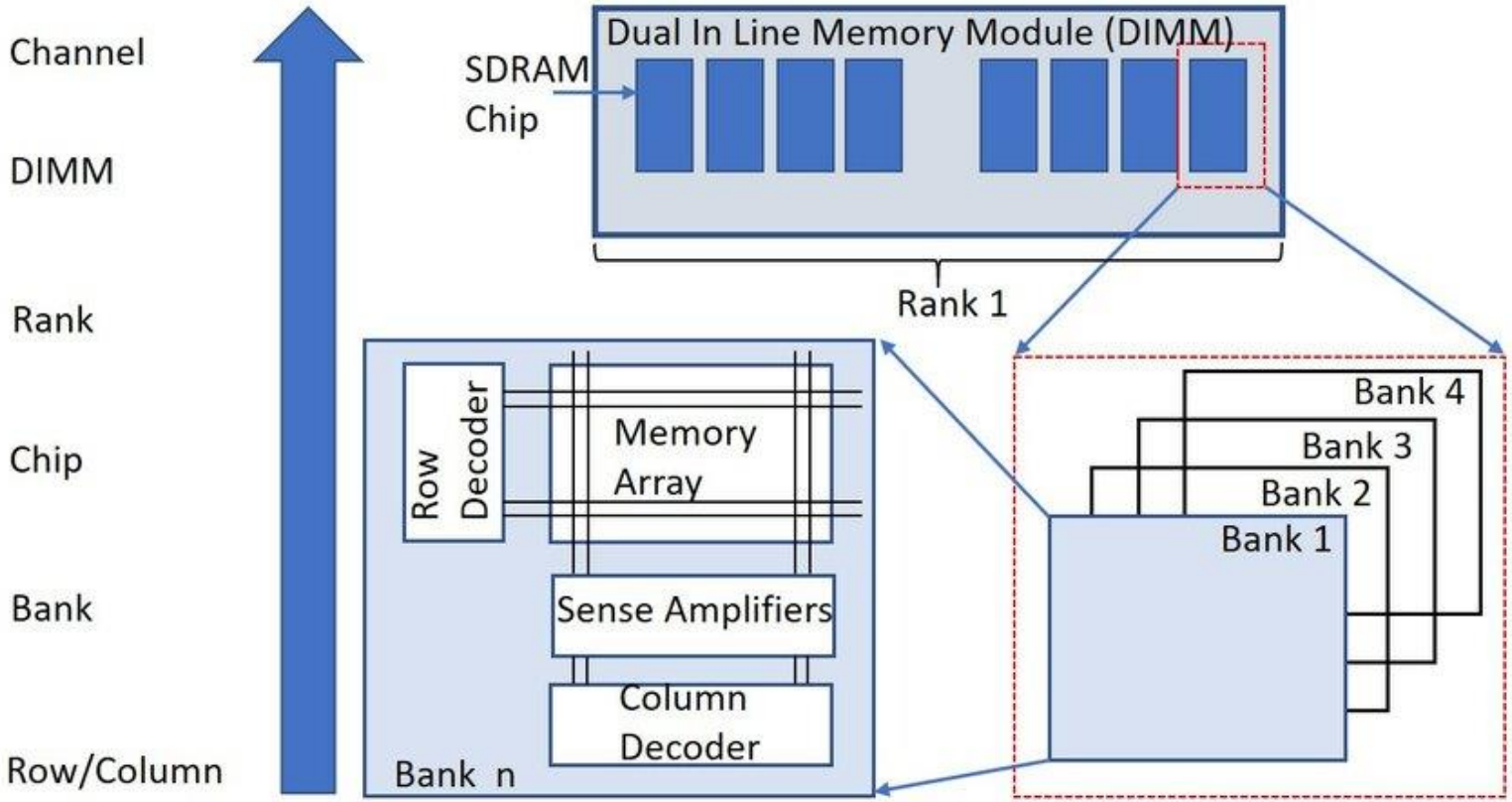


DRAM Memory Architecture



- DIMM: a PCB with DRAM chips on the back and front
- Rank: a collection of DRAM chips that work together to respond to a request and keep the data bus full
- A 64-bit data bus will need 8 x8 DRAM chips or 4 x16 DRAM chips or..
- Bank: a subset of a rank that is busy during one request
- Row buffer: the last row (say, 8 KB) read from a bank, acts like a cache

DRAM Organization and DIMM







DRAM Refresh Mechanism

- Leaky storage
- Periodic refresh across DRAM rows
- Un-accessible when refreshing
  - Variable memory latency and cache miss penalty
- Read and write the same data back
- Example:
  - 4k rows in a DRAM, 100ns read cycle, decay in 64ms
  - 4096*100ns = 410µs to refresh once
  - 410µs / 64ms = 0.64% unavailability

Memory Performance: Latency and Bandwidth

- Performance of Main Memory:
  - Latency: Cache Miss Penalty
    - *Access Time*: time between request and word arrives
    - *Cycle Time*: time between requests
  - Bandwidth: I/O & Large Block Miss Penalty (L2)
- Main Memory is *DRAM* : Dynamic Random Access Memory
  - Needs to be refreshed periodically (8 ms)
  - Addresses divided into 2 halves (Memory as a 2D matrix):
    - *RAS* or *Row Access Strobe* and *CAS* or *Column Access Strobe*
- Cache uses *SRAM* : Static Random Access Memory
  - No refresh (6 transistors/bit vs. 1 transistor)
    Size: DRAM/SRAM *4-8*
    Cost/Cycle time: SRAM/DRAM *8-16*

DDR SDRAM

Double Data Rate Synchronous DRAMs

- Burst access from a sequential locations
- Starting address, burst length
- Data transferred under control of clock
  (300 MHz, 2004)
- Clock is used to eliminate the need of synchronization and the need of supplying successive address
- Data transfer on both leading an falling edge of clock

34

Performance of DDR SDRAM

			Best case access time (no precharge)			Precharge needed
Production year	Chip size	DRAM type	RAS time (ns)	CAS time (ns)	Total (ns)	Total (ns)
2000	256M bit	DDR1	21	21	42	63
2002	512M bit	DDR1	15	15	30	45
2004	1G bit	DDR2	15	15	30	45
2006	2G bit	DDR2	10	10	20	30
2010	4G bit	DDR3	13	13	26	39
2016	8G bit	DDR4	13	13	26	39

Figure 2.4 Capacity and access times for DDR SDRAMs by year of production. Access time is for a random memory word and assumes a new row must be opened. If the row is in a different bank, we assume the bank is precharged; if the row is not open, then a precharge is required, and the access time is longer. As the number of banks has increased, the ability to hide the precharge time has also increased. DDR4 SDRAMs were initially expected in 2014, but did not begin production until early 2016.

Performance of DDR and DIMM

Standard	I/O clock rate	M transfers/s	DRAM name	MiB/s/DIMM	DIMM name
DDR1	133	266	DDR266	2128	PC2100
DDR1	150	300	DDR300	2400	PC2400
DDR1	200	400	DDR400	3200	PC3200
DDR2	266	533	DDR2-533	4264	PC4300
DDR2	333	667	DDR2-667	5336	PC5300
DDR2	400	800	DDR2-800	6400	PC6400
DDR3	533	1066	DDR3-1066	8528	PC8500
DDR3	666	1333	DDR3-1333	10,664	PC10700
DDR3	800	1600	DDR3-1600	12,800	PC12800
DDR4	1333	2666	DDR4-2666	21,300	PC21300

Figure 2.5 Clock rates, bandwidth, and names of DDR DRAMS and DIMMs in 2016. Note the numerical relationship between the columns. The third column is twice the second, and the fourth uses the number from the third column in the name of the DRAM chip. The fifth column is eight times the third column, and a rounded version of this number is used in the name of the DIMM. DDR4 saw significant first use in 2016.

Power Consumption of DDR SDRAM

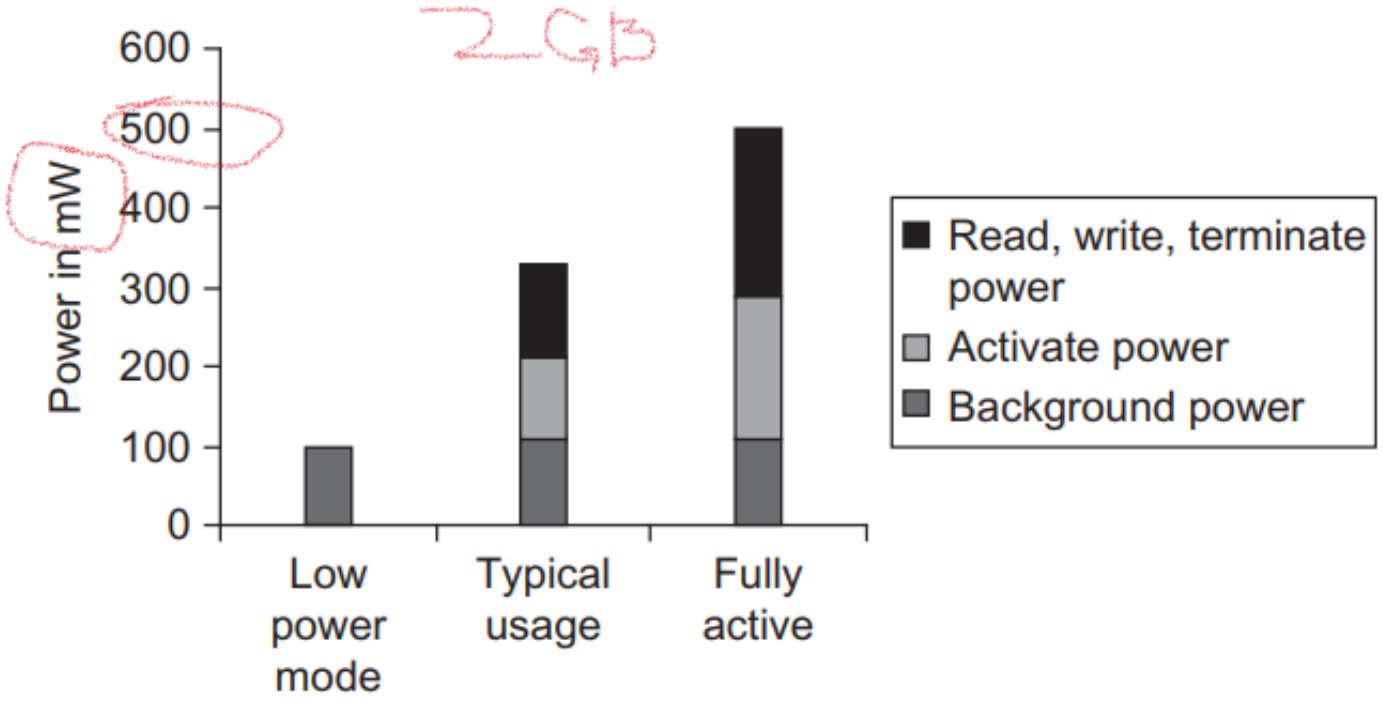


Figure 2.6 Power consumption for a DDR3 SDRAM operating under three conditions: low-power (shutdown) mode, typical system mode (DRAM is active 30% of the time for reads and 15% for writes), and fully active mode, where the DRAM is continuously reading or writing. Reads and writes assume bursts of eight transfers. These data are based on a Micron 1.5V 2GB DDR3-1066, although similar savings occur in DDR4 SDRAMs.

Comparing Cache and VM

Comparing the 2 levels of hierarchy

Cache	*Virtual Memory*
Block or Line	Page
Miss	Page Fault
Block Size: 32-64B	Page Size: 4K-16KB
Placement: Direct Mapped, N-way Set Associative	Fully Associative
Replacement: LRU or Random	Least Recently Used (LRU) approximation
Write Thru or Back	Write Back
How Managed: Hardware	Hardware + Software (Operating System)

12 199

Die Stacked DRAM

One version of this technology places the DRAM die directly on the CPU die using solder bump technology to connect them. Assuming adequate heat management, multiple DRAM dies can be stacked in this fashion. Another approach stacks only DRAMs and abuts them with the CPU in a single package using a substrate (interposer) containing the connections. Figure 2.7 shows these two different interconnection schemes. Prototypes of HBM that allow stacking of up to eight chips have been demonstrated. With special versions of SDRAMs, such a package could contain 8 GiB of memory and have data transfer rates of 1 TB/s. The 2.5D technique is currently available. Because the chips must be specifically manufactured to stack, it is quite likely that most early uses will be in high-end server chipsets.

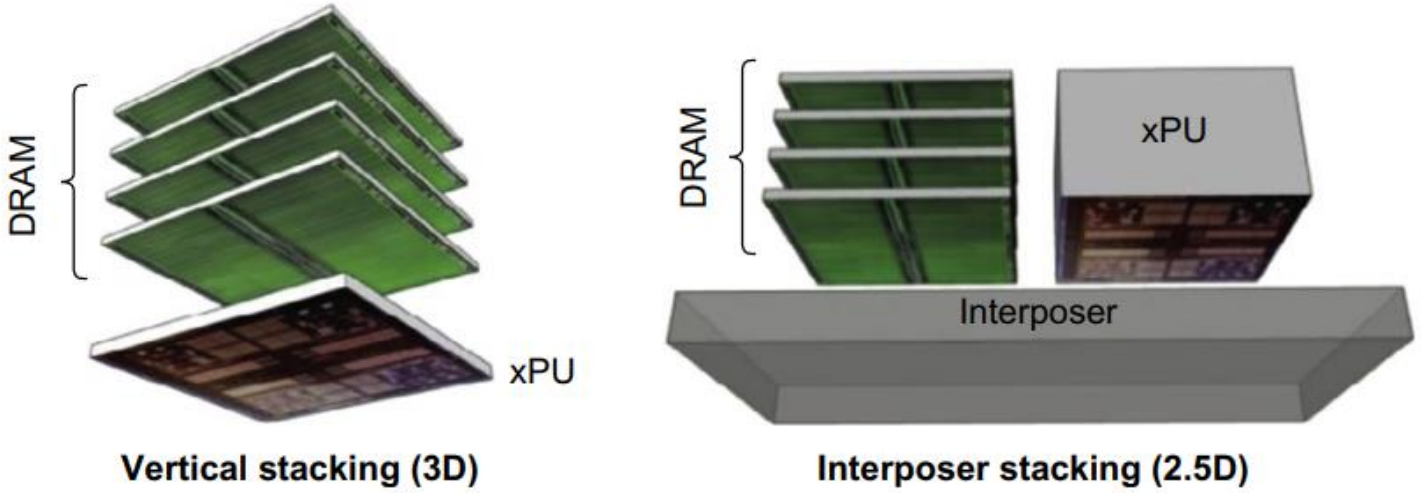
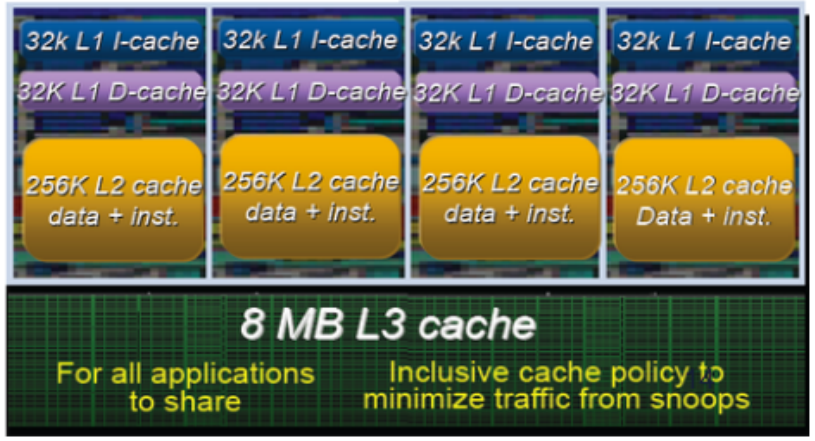


Figure 2.7 Two forms of die stacking. The 2.5D form is available now. 3D stacking is under development and faces heat management challenges due to the CPU.

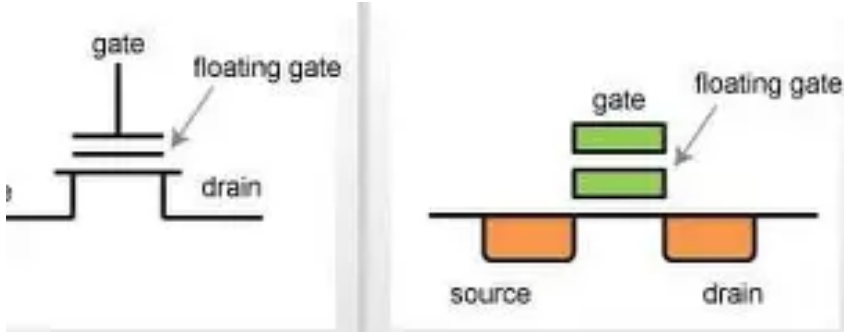
CPU – Memory Performance Gap

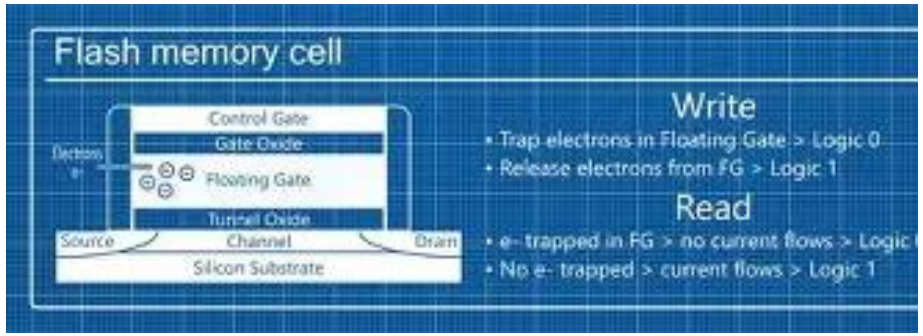
CPU-Memory Performance Gap: Bandwidth

- Memory hierarchy design becomes **more crucial with recent multi-core processors:**
- Aggregate peak bandwidth grows with # cores:
  - Intel Core i7 can generate two references per core per clock
  - Four cores and 3.2 GHz clock
    - 25.6 billion 64-bit data references/second +
    - 12.8 billion 128-bit instruction references/second
    - = 409.6 GB/s!
  - DRAM bandwidth is only 8% of this (34.1 GB/s)
- Requires:
  - Multi-port, pipelined caches
  - Two levels of cache per core
  - Shared third-level cache on chip



Flash Memory - Principle





- Type of EEPROM
- Types: NAND (denser) and NOR (faster)
- NAND Flash:
  - Reads are sequential, reads entire page (.5 to 4 KiB)
  - 25 us for first byte, 40 MiB/s for subsequent bytes
  - SDRAM: 40 ns for first byte, 4.8 GB/s for subsequent bytes
  - 2 KiB transfer: 75 uS vs 500 ns for SDRAM, 150X slower
  - 300 to 500X faster than magnetic disk

Flash Characteristics



Flash uses a very different architecture and has different properties than standard DRAM. The most important differences are

1. Reads to Flash are sequential and read an entire page, which can be 512 bytes, 2 KiB, or 4 KiB. Thus NAND Flash has a long delay to access the first byte from a random address (about 25 µS), but can supply the remainder of a page block at about 40 MiB/s. By comparison, a DDR4 SDRAM takes about 40 ns to the first byte and can transfer the rest of the row at 4.8 GiB/s. Comparing the time to transfer 2 KiB, NAND Flash takes about 75 µS, while DDR SDRAM takes less than 500 ns, making Flash about 150 times slower. Compared to magnetic disk, however, a 2 KiB read from Flash is 300 to 500 times faster. From these numbers, we can see why Flash is not a candidate to replace DRAM for main memory, but is a candidate to replace magnetic disk.
2. Flash memory must be erased (thus the name flash for the "flash" erase process) before it is overwritten, and it is erased in blocks rather than individual bytes or words. This requirement means that when data must be written to Flash, an entire block must be assembled, either as new data or by merging the data to be written and the rest of the block's contents. For writing, Flash is about 1500 times slower then SDRAM, and about 8–15 times as fast as magnetic disk.
3. Flash memory is nonvolatile (i.e., it keeps its contents even when power is not applied) and draws significantly less power when not reading or writing (from less than half in standby mode to zero when completely inactive).
4. Flash memory limits the number of times that any given block can be written, typically at least 100,000. By ensuring uniform distribution of written blocks throughout the memory, a system can maximize the lifetime of a Flash memory system. This technique, called *write leveling*, is handled by Flash memory controllers.
5. High-density NAND Flash is cheaper than SDRAM but more expensive than disks: roughly $2/GiB for Flash, $20 to $40/GiB for SDRAM, and $0.09/GiB for magnetic disks. In the past five years, Flash has decreased in cost at a rate that is almost twice as fast as that of magnetic disks.

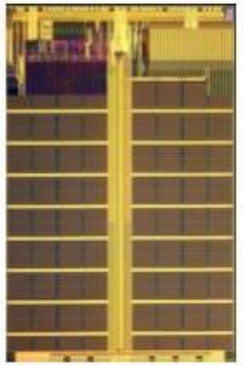
Phase Change Memory Technology

Phase-change memory (PCM) has been an active research area for decades. The technology typically uses a small heating element to change the state of a bulk substrate between its crystalline form and an amorphous form, which have different resistive properties. Each bit corresponds to a crosspoint in a two-dimensional network that overlays the substrate. Reading is done by sensing the resistance between an x and y point (thus the alternative name *memristor*), and writing is accomplished by applying a current to change the phase of the material. The absence of an active device (such as a transistor) should lead to lower costs and greater density than that of NAND Flash.

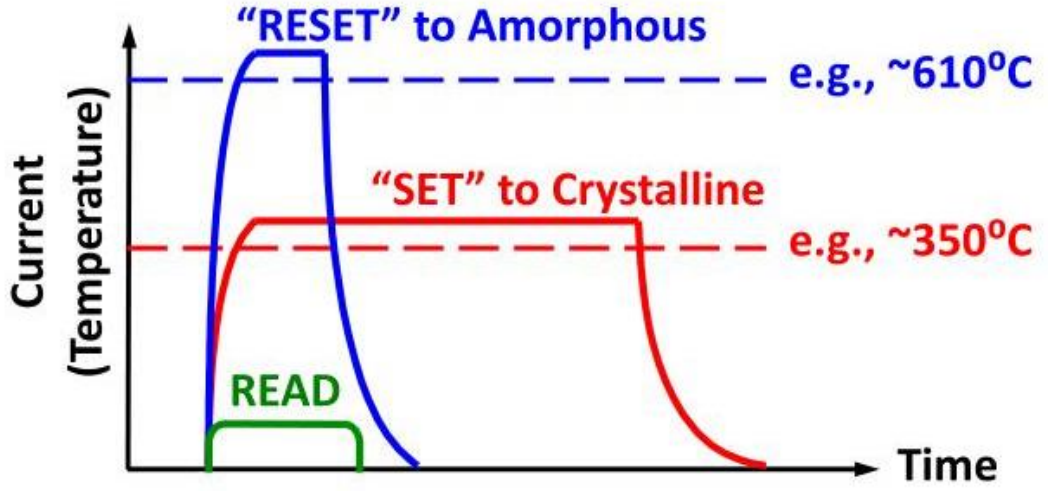
In 2017 Micron and Intel began delivering Xpoint memory chips that are believed to be based on PCM. The technology is expected to have much better write durability than NAND Flash and, by eliminating the need to erase a page before writing, achieve an increase in write performance versus NAND of up to a factor of ten. Read latency is also better than Flash by perhaps a factor of 2–3. Initially, it is expected to be priced slightly higher than Flash, but the advantages in write performance and write durability may make it attractive, especially for SSDs. Should this technology scale well and be able to achieve additional cost reductions, it may be the solid state technology that will depose magnetic disks, which have reigned as the primary bulk nonvolatile store for more than 50 years.

Phase Change Memory (PCM) Principle

Phase Change Memory (PCM)

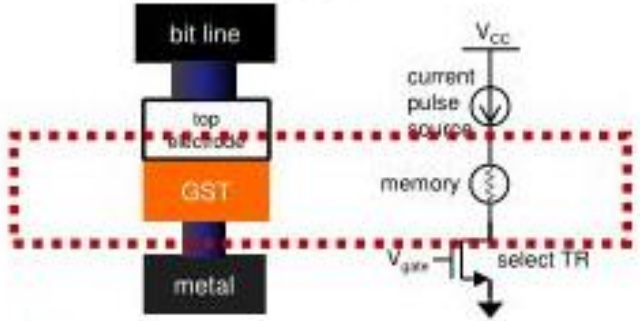


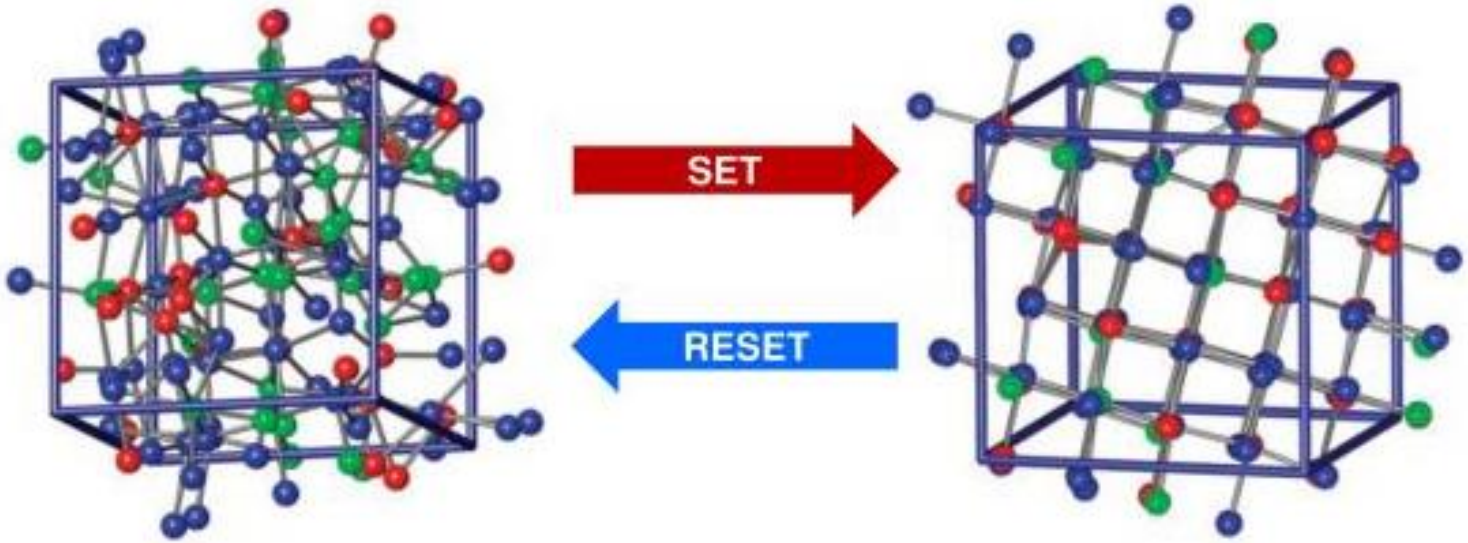
- Byte-addressable non-volatile memory
- Two states of phase change material:
  - Amorphous: high resistance, representing "0"
  - Crystalline: low resistance, representing "1"
- Operations:



PCRAM States Operation

Phase-change RAM (PRAM)





Amorphous = high resistivity

Crystalline = low resistivity

(Pictures from Hegedüs and Elliott, *Nature Materials*, March 2008)

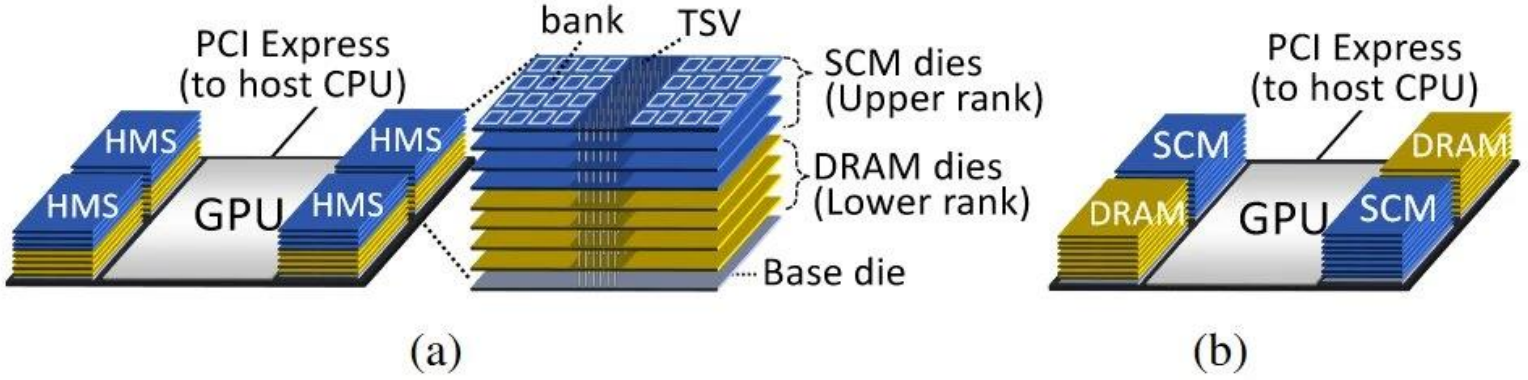
Graphics Data SDRAM

GDRAMs or GSDRAMs (Graphics or Graphics Synchronous DRAMs) are a special class of DRAMs based on SDRAM designs but tailored for handling the higher bandwidth demands of graphics processing units. GDDR5 is based on DDR3 with earlier GDDRs based on DDR2. Because graphics processor units (GPUs; see Chapter 4) require more bandwidth per DRAM chip than CPUs, GDDRs have several important differences:

1. GDDRs have wider interfaces: 32-bits versus 4, 8, or 16 in current designs.
2. GDDRs have a higher maximum clock rate on the data pins. To allow a higher transfer rate without incurring signaling problems, GDRAMS normally connect directly to the GPU and are attached by soldering them to the board, unlike DRAMs, which are normally arranged in an expandable array of DIMMs.

Altogether, these characteristics let GDDRs run at two to five times the bandwidth per DRAM versus DDR3 DRAMs.

Modern GPU Memory Organization



TSV = Through Hole Via
SCM = Storage Class Memory

Fig. 5. Design space of a GPU with SCM and DRAM cache with (a) 3D-stacked DRAM and SCM and (b) separate DRAM and SCM stacks.

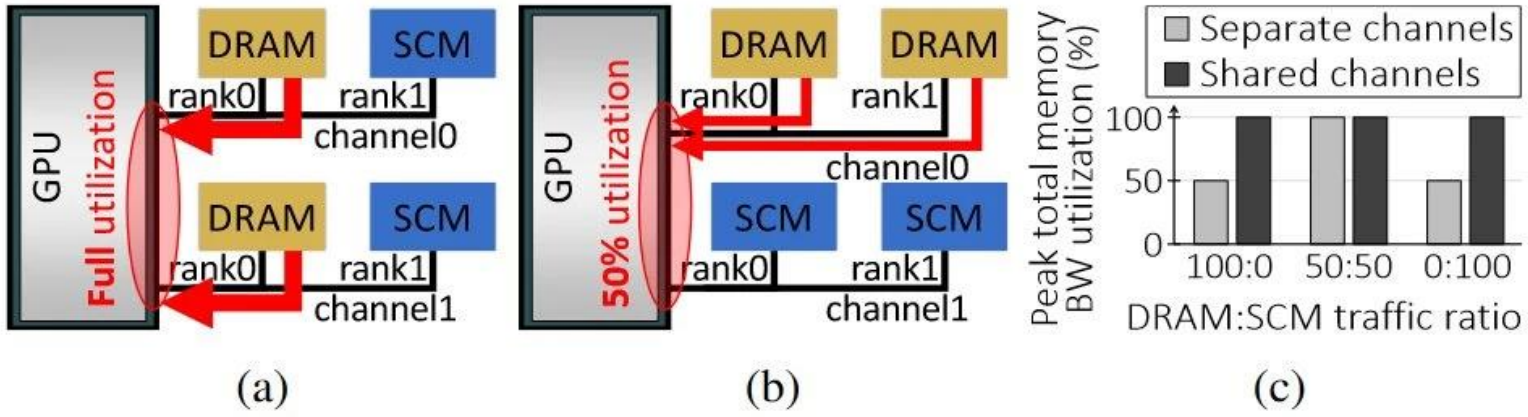
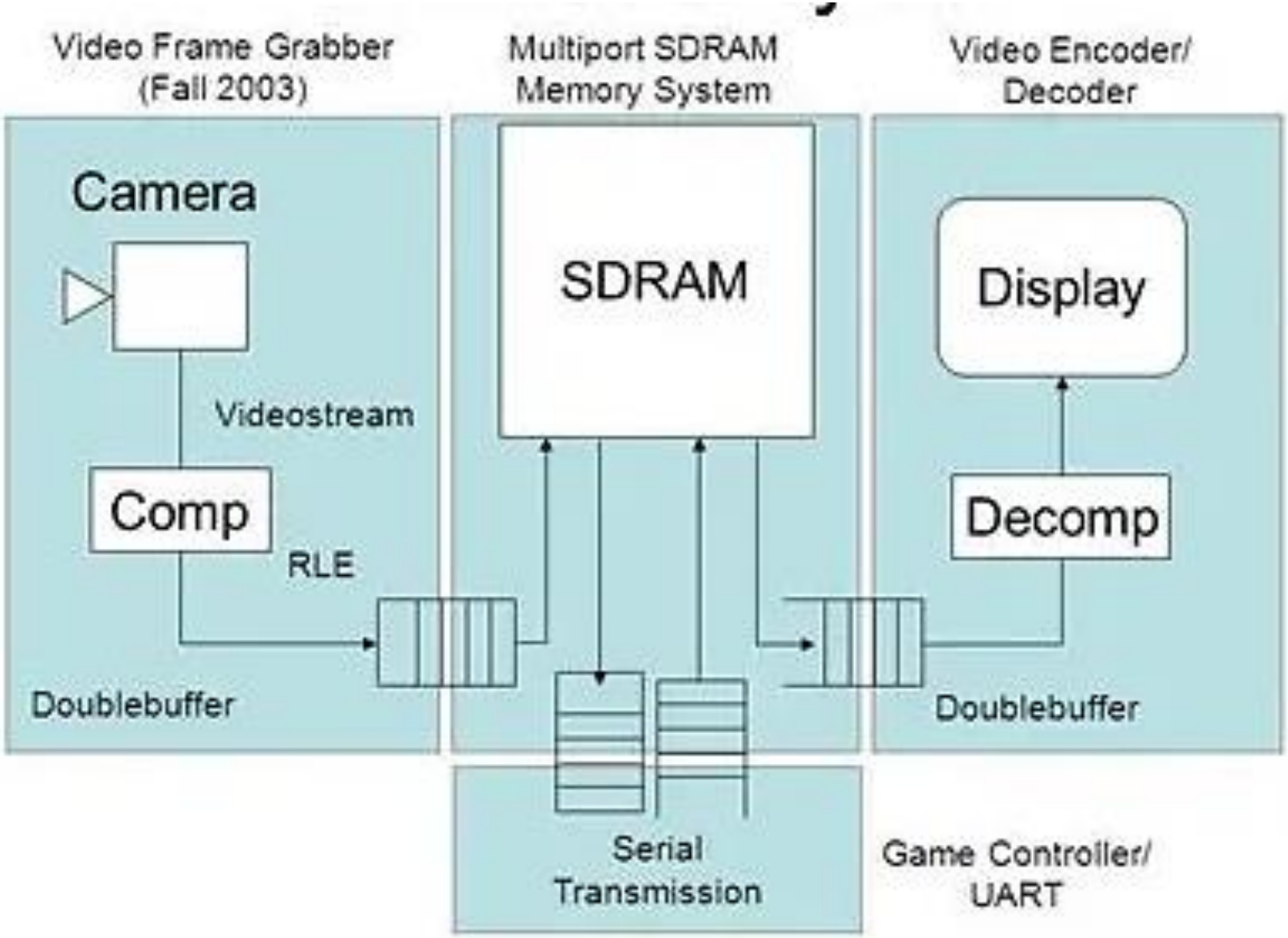


Fig. 6. Integration of SCM and DRAM cache using (a) shared channels and (b) separate channels. (c) Peak memory BW for varying DRAM-to-SCM traffic ratios.

Multiport Memory for High Throughput



Dependability in Memory Systems

Enhancing Dependability in Memory Systems

Large caches and main memories significantly increase the possibility of errors occurring both during the fabrication process and dynamically during operation. Errors that arise from a change in circuitry and are repeatable are called *hard errors* or *permanent faults*. Hard errors can occur during fabrication, as well as from a circuit change during operation (e.g., failure of a Flash memory cell after many writes). All DRAMs, Flash memory, and most SRAMs are manufactured with spare rows so that a small number of manufacturing defects can be accommodated by programming the replacement of a defective row by a spare row. Dynamic errors, which are changes to a cell's contents, not a change in the circuitry, are called *soft errors* or *transient faults*.

Dynamic errors can be detected by parity bits and detected and fixed by the use of error correcting codes (ECCs). Because instruction caches are read-only, parity suffices. In larger data caches and in main memory, ECC is used to allow errors to be both detected and corrected. Parity requires only one bit of overhead to detect a single error in a sequence of bits. Because a multibit error would be undetected with parity, the number of bits protected by a parity bit must be limited. One parity bit per 8 data bits is a typical ratio. ECC can detect two errors and correct a single error with a cost of 8 bits of overhead per 64 data bits.

Error Correction Codes

Caches and TLBs

https://www.cs.nthu.edu.tw/~king/courses/cs5100/2014/L06-Memory.pdf

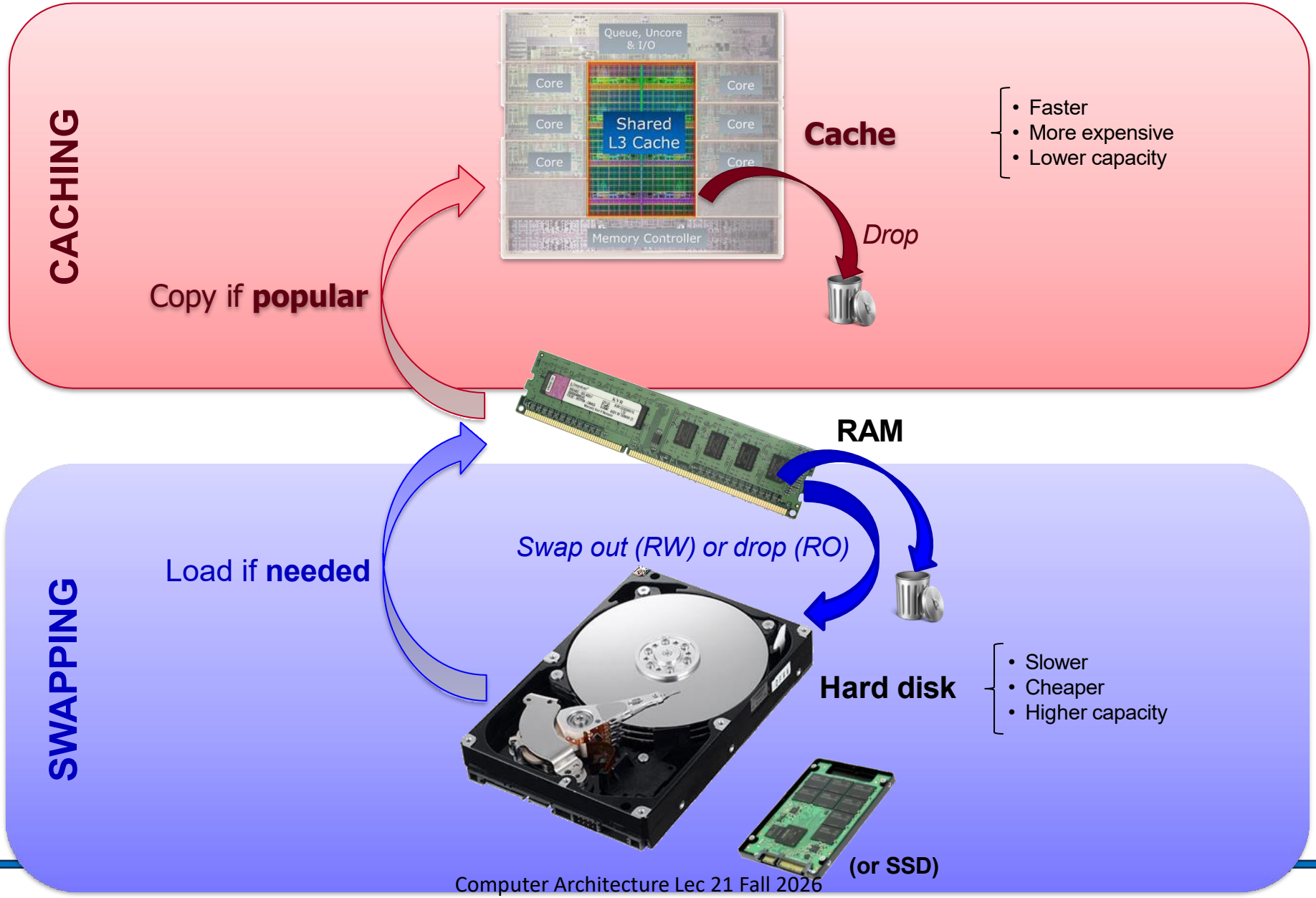
Cache Optimizations through Lower Miss Rate

Discussed Previously

Six Basic Cache Optimizations

- Larger block size
  - Exploit spatial locality
  - Reduce compulsory misses
  - Increase capacity and conflict misses, increase miss penalty and false sharing
- Larger total cache capacity
  - Reduce capacity misses
  - Increase hit time, increase power consumption
- Higher associativity
  - Reduce conflict misses
  - Increase hit time, increase power consumption

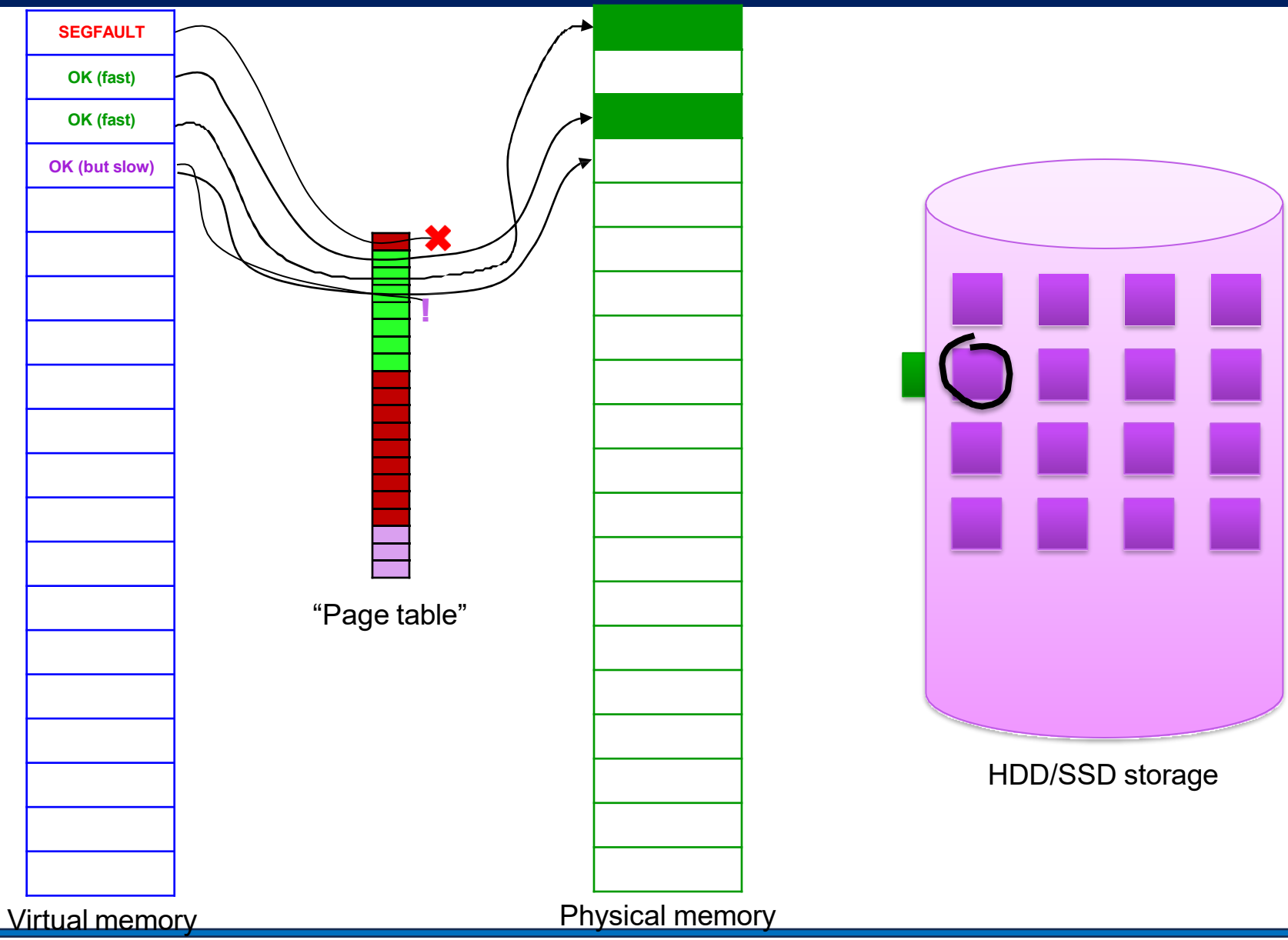
Figure: Caching vs. Virtual Memory



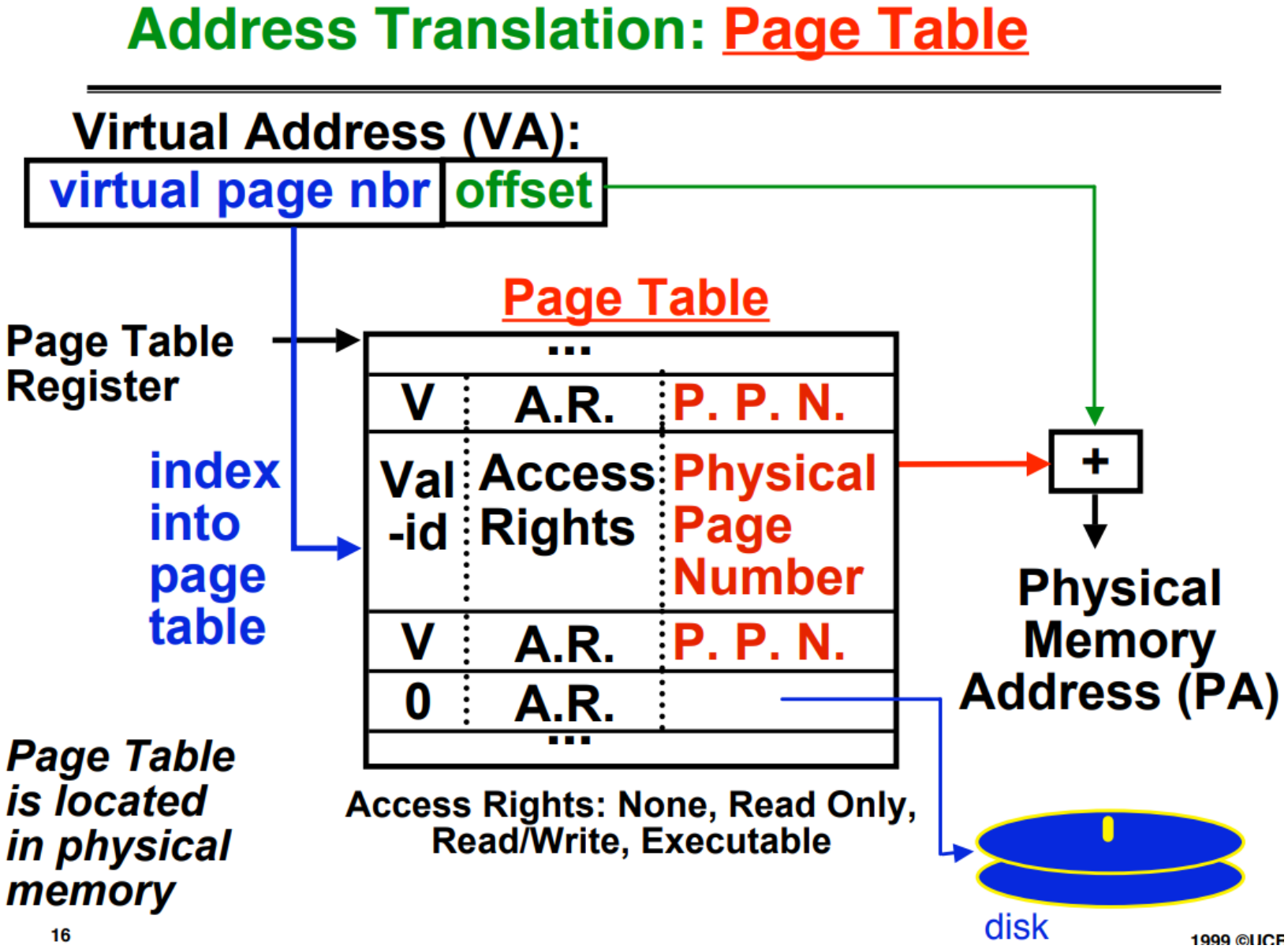
Logical vs Physical Addresses

- The concept of a logical address space that is bound to a separate physical address space is central to proper memory management
  - **Logical** (**virtual**) **address** – as generated by the CPU
  - **Physical address** – address seen by the memory unit
  - Identical in compile-time and load-time address-binding schemes
  - Differ in execution-time address-binding schemes
- The logical/physical address space is the set of all logical/physical addresses generated by a program
- Need hardware support to perform the mapping from logical to physical addresses at run time

High Level VM Operation

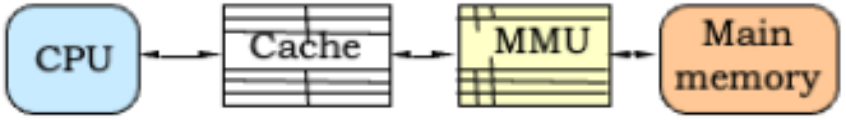


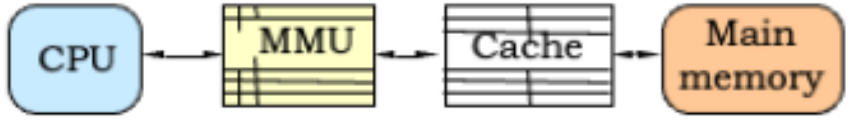
Address Translation in Page Table



Using Caches with Virtual Memory

Virtually-Addressed Cache	*Physically-Addressed* Cache
Tags from virtual addresses	Tags from physical addresses



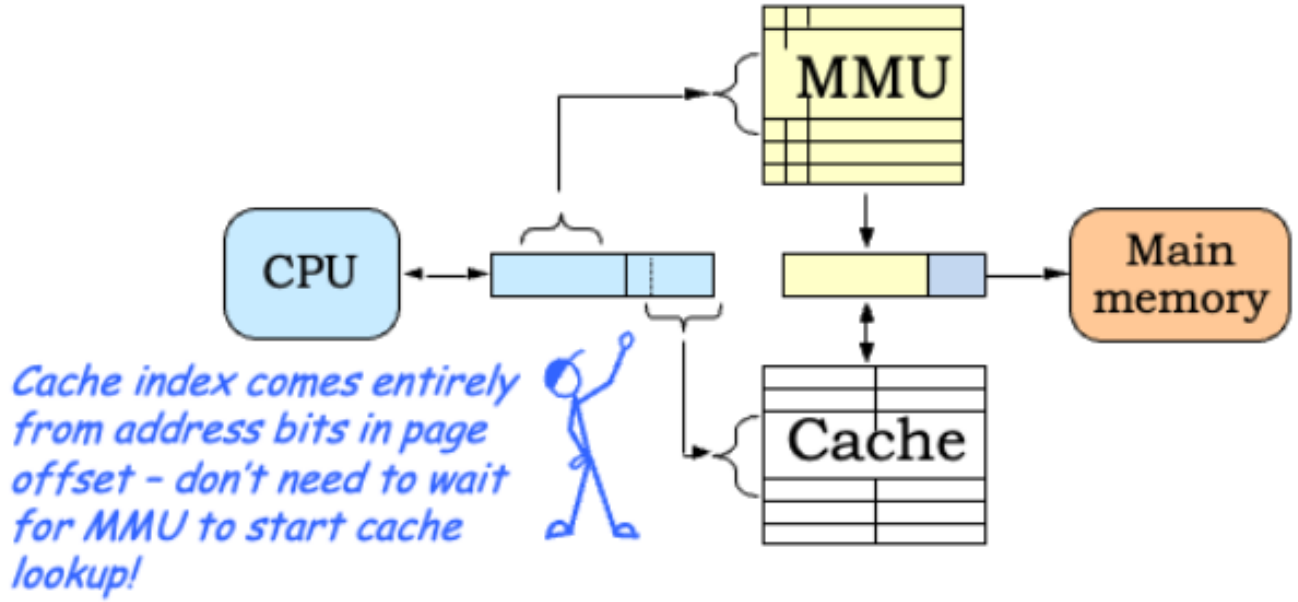


- FAST: No MMU time on HIT
- Problem: Must flush cache after context switch

- Avoids stale cache data after context switch
- SLOW: MMU time on HIT

Overlap Operation of Cache and TLB

Best of Both Worlds: Overlapped Operation



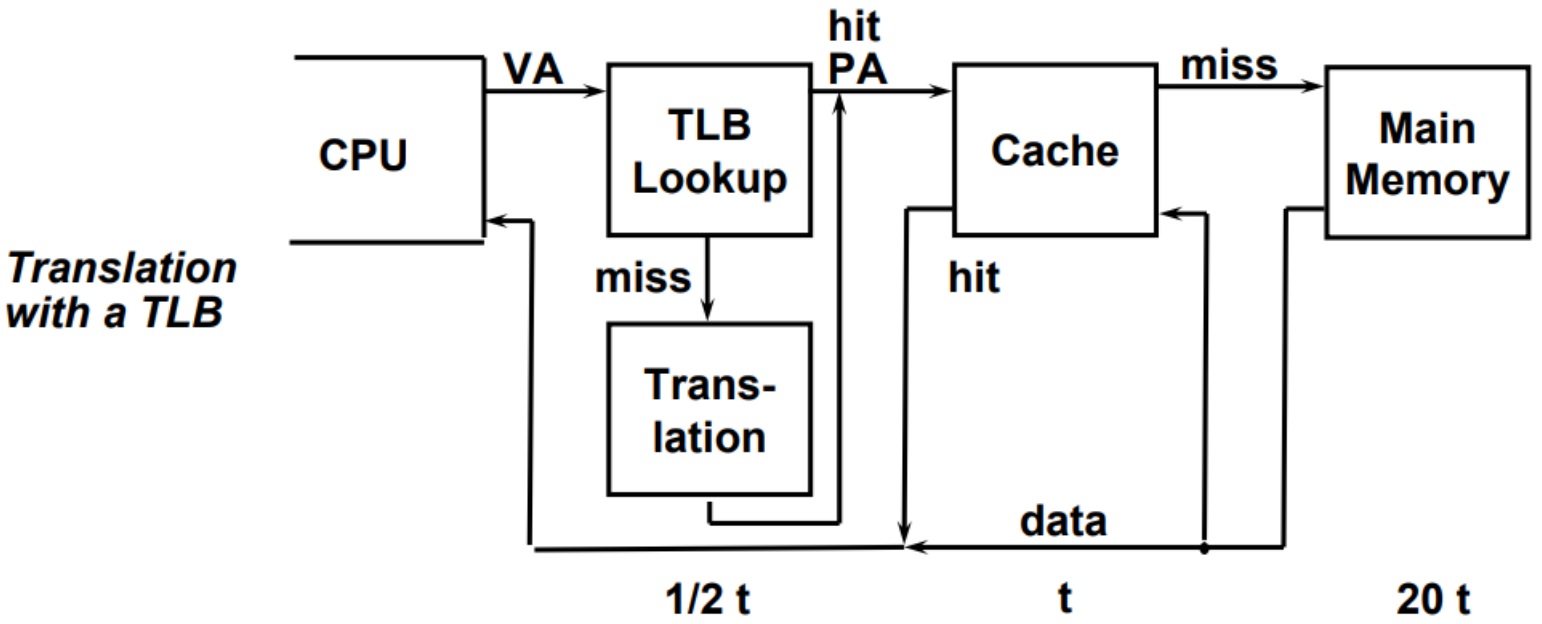
OBSERVATION: If cache index bits are a subset of page offset bits, tag access in a physical cache can *overlap* page map access. Tag from cache is compared with physical page address from MMU to determine hit/miss.

Problem: Limits # of bits of cache index → increase cache capacity by increasing associativity

Practical Approach for Address Translation

Making Address Translation Practical

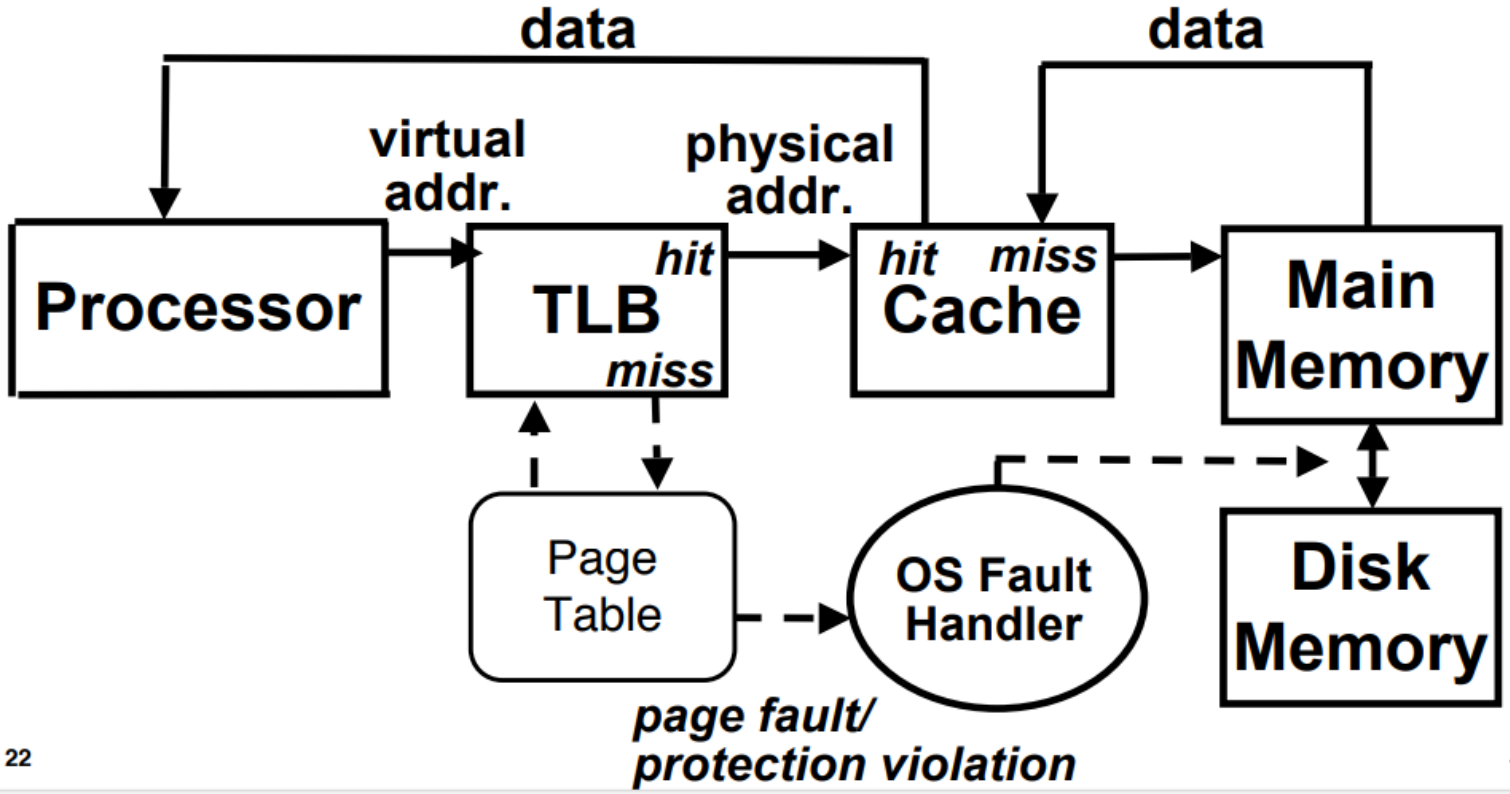
- In VM, memory acts like a cache for disk
  - Page table maps virtual page numbers to physical frames
  - Use a page table cache for recent translation
    => *Translation Lookaside Buffer* (TLB)



TLB and Cache Together – Another Design

Translation Look-Aside Buffers

- TLB is usually small, typically 32-4,096 entries
- Like any other cache, the TLB can be fully associative, set associative, or direct mapped



22

1999

Translation Lookaside Buffer

- Typical RISC processors have *memory management unit* (MMU) which includes TLB and does page table lookup
  - TLB can be organized as fully associative, set associative, or direct mapped
  - TLBs are small, typically < 128 - 256 entries
    - Fully associative on high-end machines, small n-way set associative on mid-range machines
- TLB hit on write:
  - Toggle dirty bit (write back to page table on replacement)
- TLB miss:
  - If only TLB miss => load PTE into TLB (SW or HW?)
  - If page fault also => OS exception

73

TLB of MIPS R2000

- 4KB pages, 32-bit VA
  => virtual page number: 20 bits
- TLB organization:
  - 64 entries, fully assoc., serve instruction and data
  - 64-bit/entry (20-bit tag, 20-bit physical page number, valid, dirty)
- On TLB miss:
  - Hardware saves page number to a special register and generates an exception
  - TLB miss routine finds PTE, uses a special set of system instructions to load physical addr into TLB
- Write requests must check a write access bit in TLB to see if it has permit to write
  => if not, an exception occurs

Integrating TLB and Cache

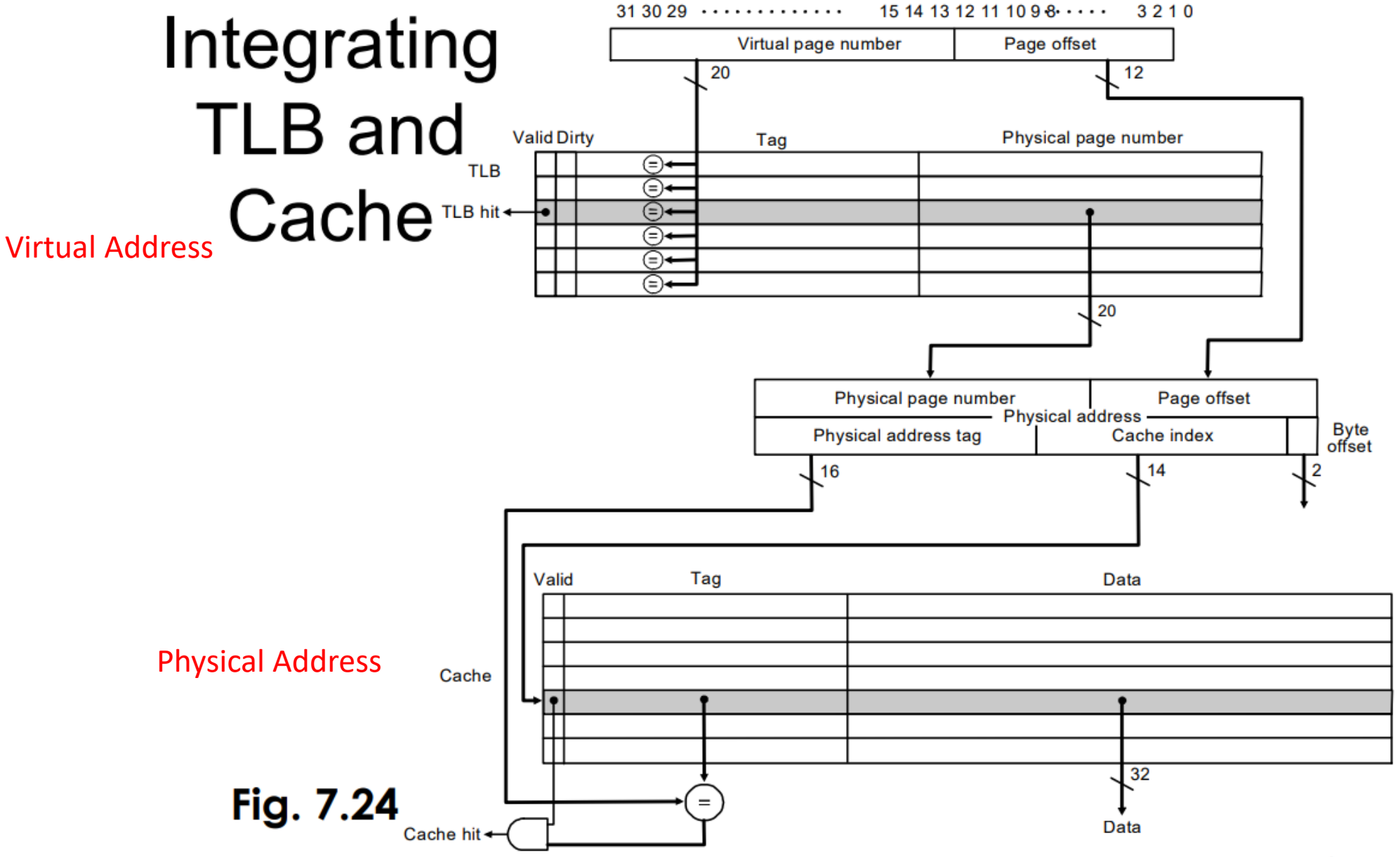
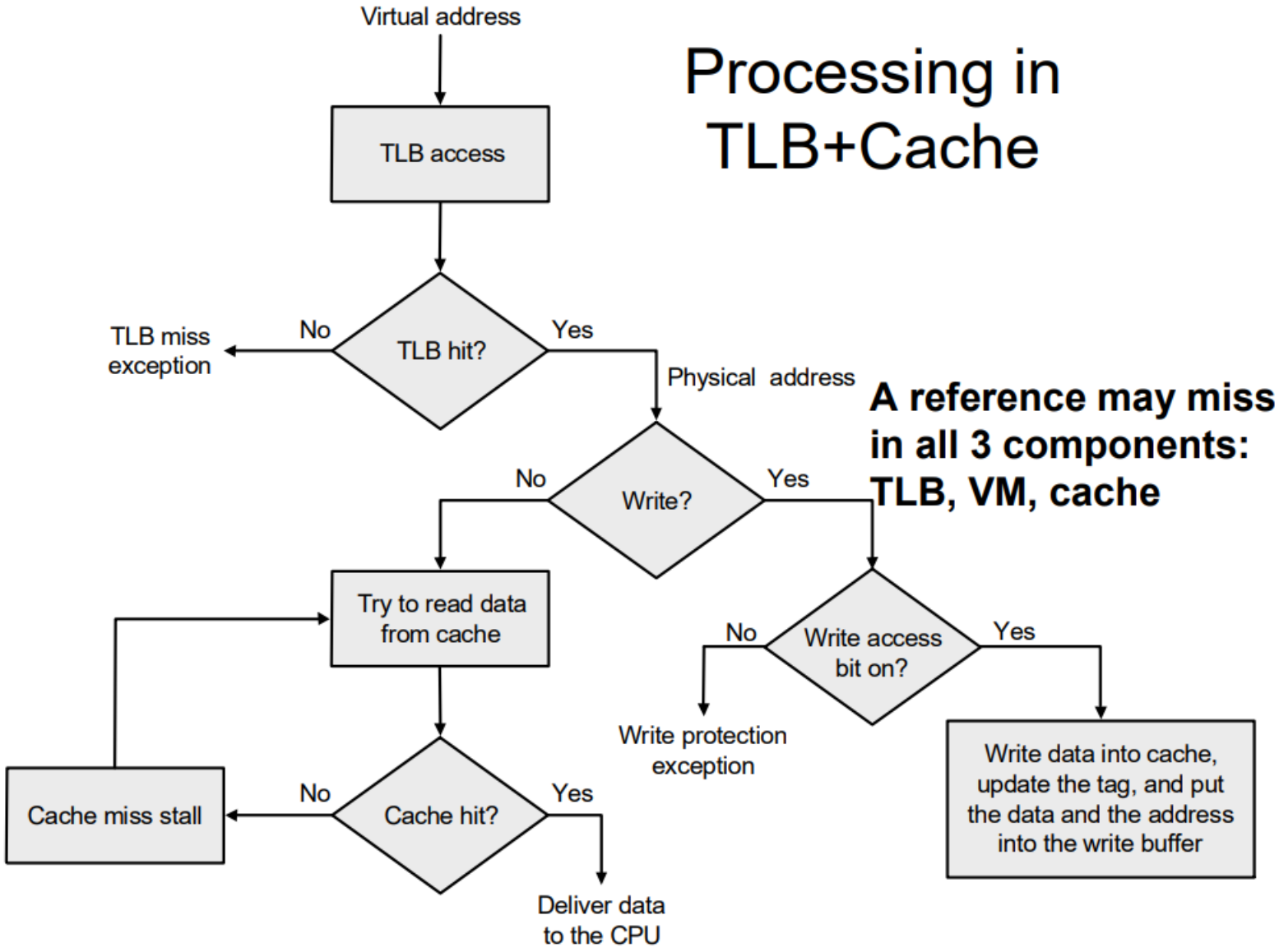


Fig. 7.24

Processing TLB + Cache



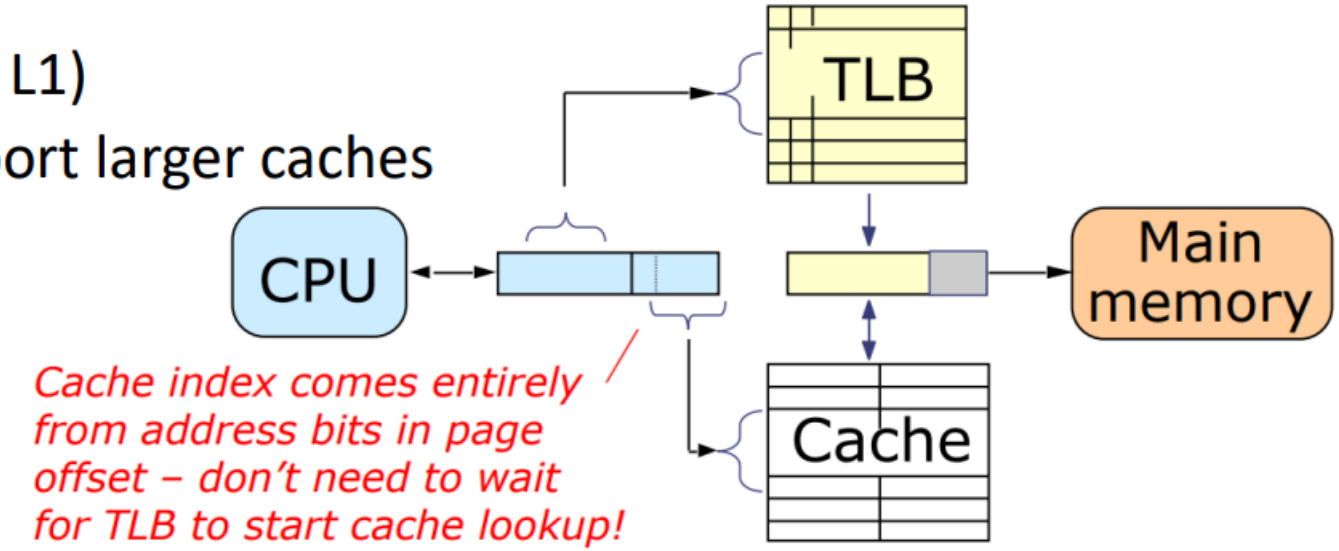
Possibilities in TLB + Cache

Possible Combinations of Events

Cache	TLB	Page table	Possible? Conditions?
Miss	Hit	Hit	Yes; but page table never checked if TLB hits
Hit	Miss	Hit	TLB miss, but entry found in page table;after retry, data in cache
Miss	Miss	Hit	TLB miss, but entry found in page table; after retry, data miss in cache
Miss	Miss	Miss	TLB miss and is followed by a page fault; after retry, data miss in cache
Miss	Hit	Miss	impossible; not in TLB if page not in memory
Hit	Hit	Miss	impossible; not in TLB if page not in memory
Hit	Miss	Miss	impossible; not in cache if page not in memory

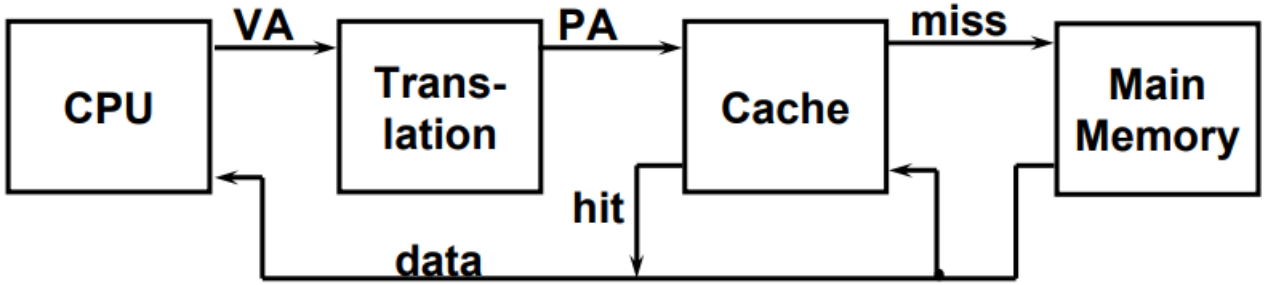
Mixed physical and virtual caching

- Another solution: mixing virtual and physical addressing
- Virtually-Indexed, Physically Tagged (VIPT) cache
  - **IF** (cache index bit width ≤ page offset bit width)
    TLB and cache lookups can be done in parallel
    - Cache lookup done only using page offset, which is not translated by virtual memory
      -> TLB lookup overhead is hidden!
  - Only works for small caches (e.g., L1)
  - Can increase associativity to support larger caches

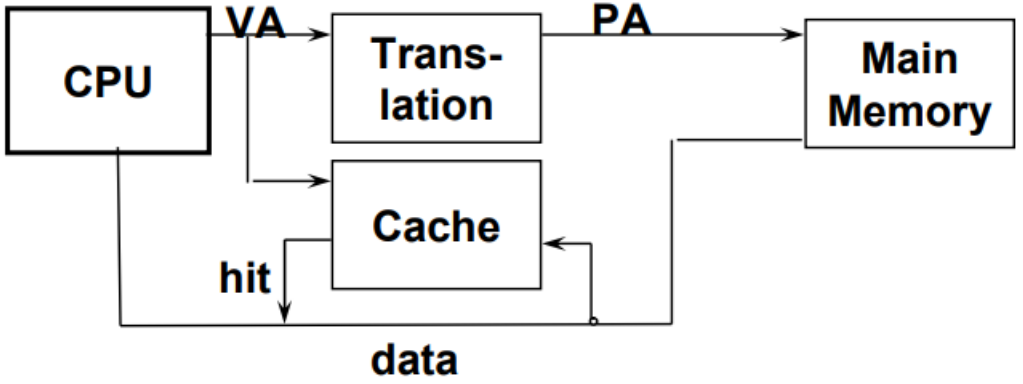


Virtual Address and Cache

- TLB access is serial with cache access
  - Cache is physically indexed and tagged



- Alternative: *virtually addressed cache*
  - Cache is virtually indexed and virtually tagged



Virtually Addressed Cache

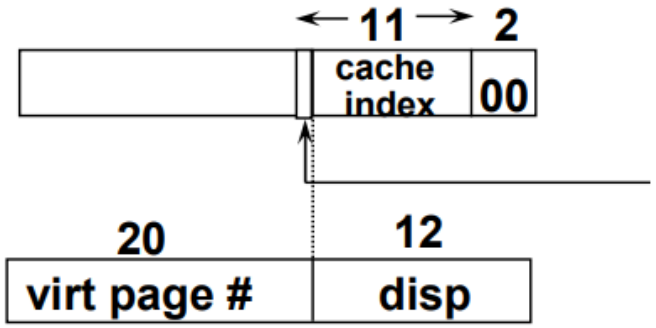
- Require address translation only on miss!
- Problem:
  - Same virtual addresses (different processes) map to different physical addresses: tag + process id
  - *Synonym/alias problem*: two different virtual addresses map to same physical address
    - Two different cache entries holding data for the same physical address!
  - For update: must update all cache entries with same physical address or memory becomes inconsistent
  - Determining this requires significant hardware, essentially an associative lookup on the physical address tags to see if you have multiple hits;
  - Or software enforced alias boundary: same least-significant bits of VA &PA > cache size

80

Problem with Overlap Access

Problem with Overlapped Access

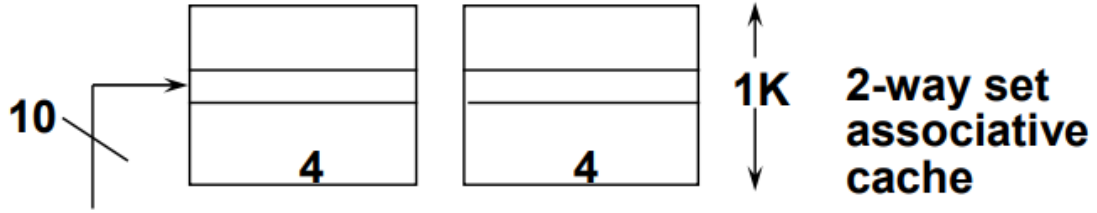
- Address bits to index into cache must not change as a result of VA translation
  - Limits to small caches, large page sizes, or high n-way set associativity if want a large cache
  - Ex.: cache is 8K bytes instead of 4K:



This bit is changed by VA translation, but is needed for cache lookup

Solutions:

go to 8K byte page sizes;
go to 2 way set associative cache
SW guarantee VA[13]=PA[13]

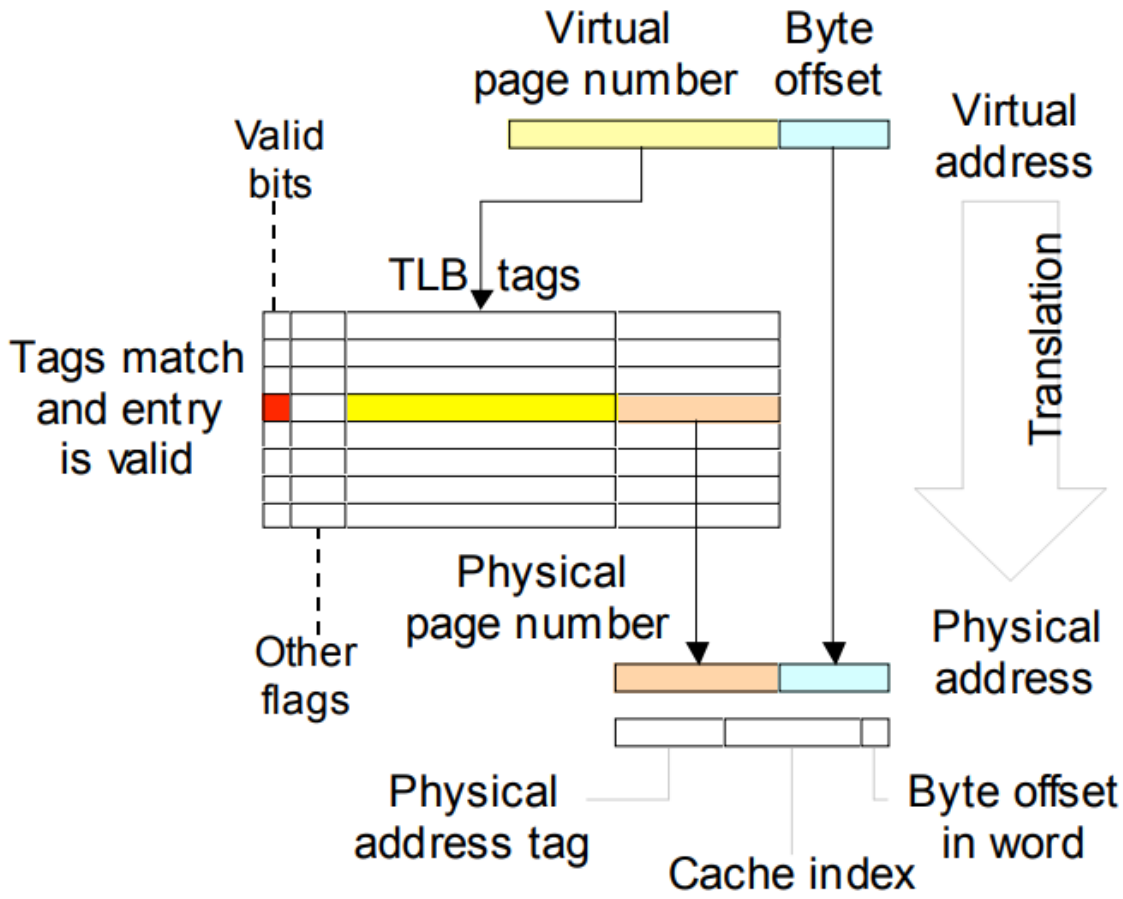


TLB and Caches

- Several design alternatives
  - PIPT: Physically-indexed Physically-tagged Cache
  - VIVT: Virtually-indexed Virtually-tagged Cache
  - VIPT: Virtually-indexed Physically-tagged Cache
  - PIVT: Physically-indexed Virtually-tagged Cache
    - Not outright useful, R6000 is the only that used this

Address Translation for Caches

Translation Lookaside Buffer



Virtual-to-physical address translation by a TLB and how the resulting physical address is used to access the cache memory.

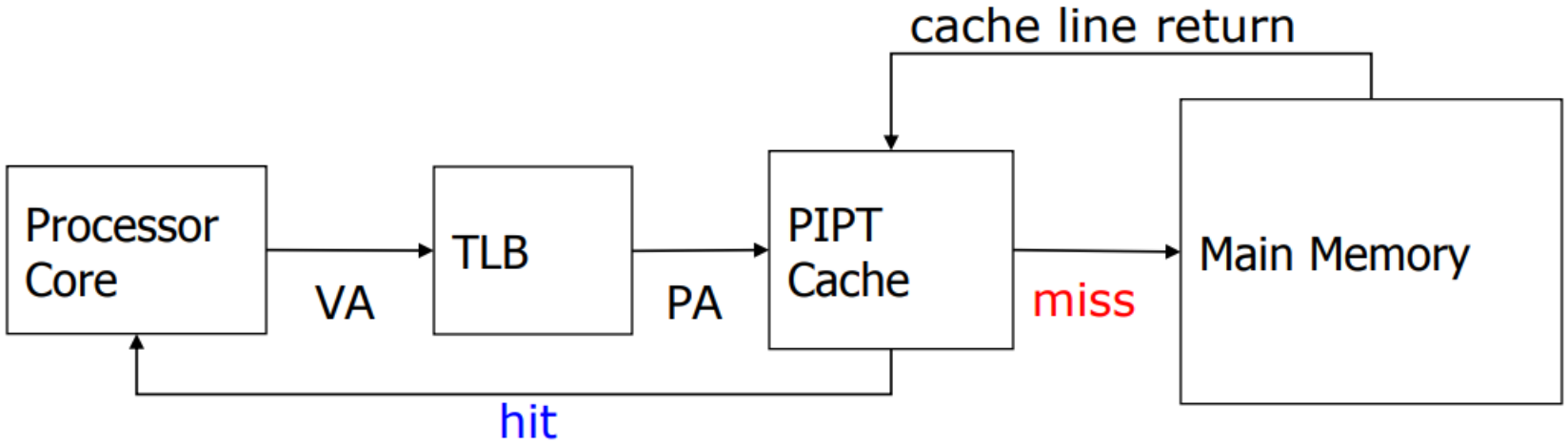
Physically Indexed Physically Tagged Cache

- PIPT caches are typically slow
  - We can't know the (physical) address to cache before going through virtual memory translation
    - Every cache access requires a virtual -> physical translation!
  - No cache benefits if TLB miss
  - TLB hits still result in performance loss because TLB lookup also has latency
- Benefits of PIPT
  - Simple to implement, as cache is unaware of virtual memory
  - Shared memory between processes are handled automatically
    - Same physical memory mapped to virtual memories of different processes…

Q: Is this a problem for caches larger than L1?

Physically Indexed Physically Tagged PIPT

Physically-Indexed Physically-Tagged (PIPT)



- Slower, always translate address before accessing memory
- Simpler for data coherence

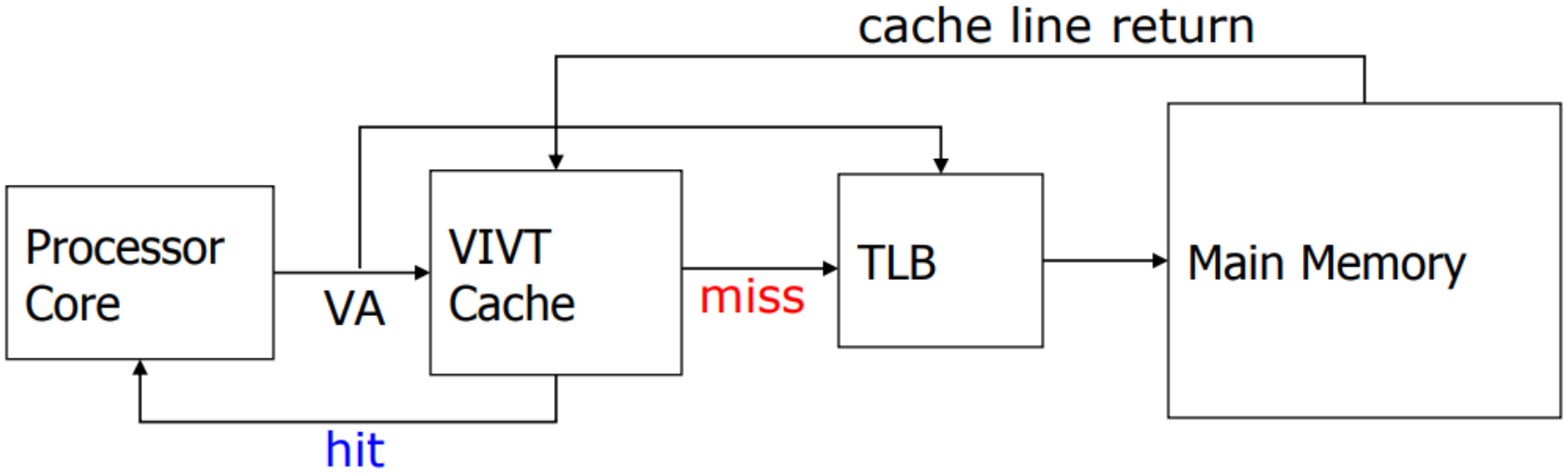
Virtually Indexed Virtually Tagged VIVT

Virtually Indexed Virtually Tagged Cache

- One solution: "Address Space ID"
  - Virtual address can be augmented with a small, process-specific ASID (Address-Space ID), so that TLB entries from different processes can be distinguished
  - Pros: High performance!
  - Cons: Requires additional hardware and OS support
  - Cons: May limit number of concurrent processes
    - E.g., ARM sometimes uses 8-bit ASIDs -> 256 concurrent processes?!
    - With more, caches are invalidated, ASID allocation starts over (By Linux!)

Virtually Indexed Virtually Tagged VIVT

Virtually-Indexed Virtually-Tagged (VIVT)



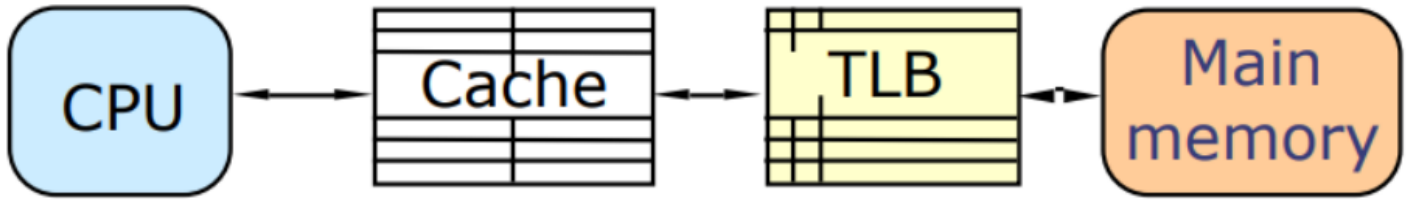
- Fast cache access
- Only require address translation when going to memory (miss)
- Issues?

VIVT Explained

Virtually Indexed Virtually Tagged Cache

VIVT

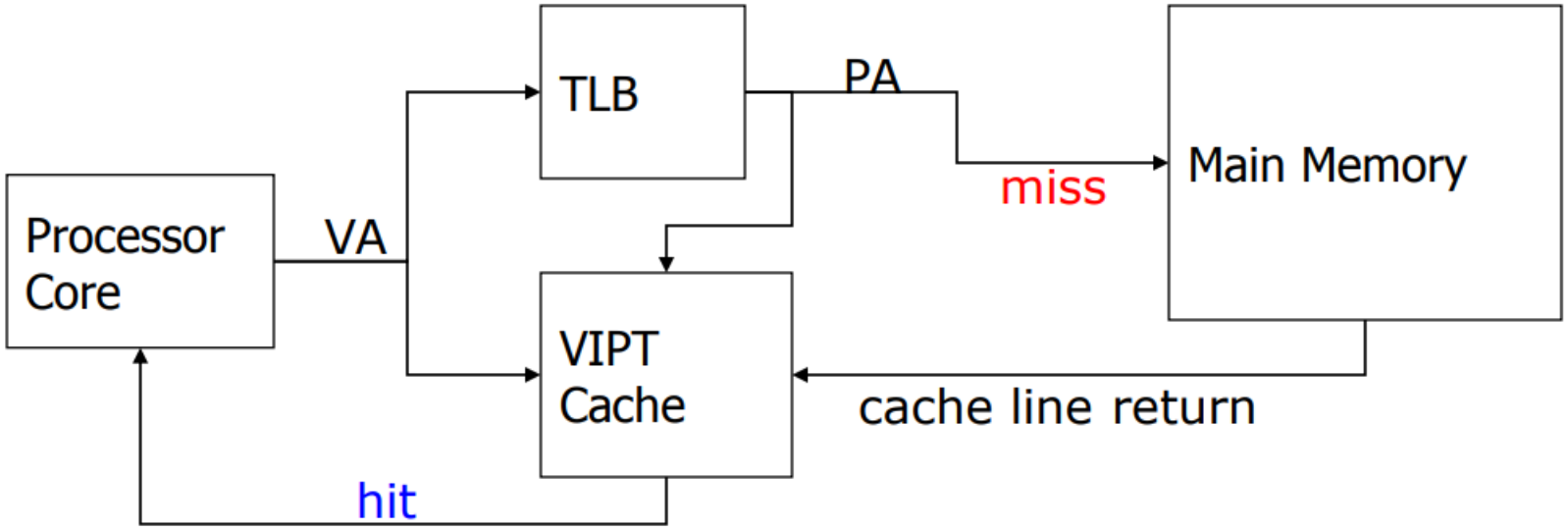
- If cache is based on virtual address
  - Cache hits are fast! No translation required before access
  - But, different processes can and will be using the same virtual memory address referring to different physical locations ("Homonyms problem")
    - Low performance is bad, but INCORRECT BEHAVIOR IS NOT ACCEPTABLE!
  - Naïve solution: Cache flushed after every context switch for correctness
    - Compulsory misses after every context switch… Performance hit!
  - Also, shared physical memory can be cached twice or more



"Virtually Indexed, Virtually Tagged (VIVT)"

Virtually Indexed Physically Tagged VIPT

Virtually-Indexed Physically-Tagged (VIPT)



- Gain benefit of a VIVT and PIPT
- Parallel access to TLB and VIPT cache
- No homonym
- How about synonym?

TLB and Caches Design Choices

TLB and Caches

- ~~Several design alternatives~~
  - ~~PIPT: Physically-indexed Physically-tagged Cache~~
  - ~~VIVT: Virtually-indexed Virtually-tagged Cache~~
  - ~~VIPT: Virtually-indexed Physically-tagged Cache~~
  - PIVT: Physically-indexed Virtually-tagged Cache
    - Not outright useful, R6000 is the only that used this

